

```

#####
@kernel-nr: lex | System Handbuch | kernel lex | 00 FallBack | #####
#####
# | - nicht freigeschaltet | + Freigeschaltet | X BauStelle | #
#-----#
# - SOFTWARE MCS Kernel - #
# 01 X Datentypen -fix2 13 - Warp -fix #
# 02 - Transaktion -fix1 14 X p Identifikation -fix2 #
# 03 - Protein -fix4 15 - Argumente -fix #
# 04 - Proton -fix 16 - SchaltVerknüpfungen #
# 05 - Trennbefehl -fix2 17 - Adressierung -new #
# 06 - Aktion -fix2 18 X AI Synctimer #
# 07 - Eingaben -fix4 #
# 08 - Wahrheiten -fix4 20 - Reihenfolge #
# 09 - Operatoren -fix4 77 X Ausführungslogik -fix2 #
# 10 - Feed -fix2 88 - Dateitypen -fix #
# 11 - Zielreferenz 99 - Register CMDs -new #
# 12 - Dirigent 100 - BMEotionen #
#####
# - MCSPLUS Programmiersprache - #
# 101 - Einführung 102 - Grundeinheit #
# 103 - Tagging und Rollenbindung #
#####
# - HARDWARE Komponenten BrainMaschineCore BMC - #
# 800 - ChiPBezeichnungen 803 - MIT-Taktgeber #
# 801 - EnergieMatrix 804 - MBMatrix #
# 802 - Lichtleiterbahnen #
#####
# - LICENSES GELTUNGSBEREICH - #
# 900 L0_VorWort 993 L1_CoreKernelMandat #
# 994 L2_AuthorRights 995 L3_CommercialLicensing #
# 996 L4_AI-AdministrationMandate 997 L5_Post_Mortem-Control #
# 998 L6_immunitable_Cores_definition 999 License_Manifest #
#####
# System | SOFTWARE MCS KERNEL | HARDWARE BrainMaschineCore #
# Tastaturstandart: UTF -8 Deutscher Standart #
# Single Source of Truth (SSoT) Lexikon Version 00.41 #
# MCS 3.7 | MCSPLUS 0.2 | BMC Version 0.4 | License Version 1.0 #
# Autor: Thomas Zimmermann 23.04.1988 (Stufe 10) (p7F3A) #
#-----#
# Standort: /Pylovara/Handbuch/KernelNotes/ Germany #
# Status: Vollständige Rollende-Verbesserungen Pylovara© #
# PDF NamensLayer: Pylovara-System-XX.XX.pdf since 2025 #
#####

```

@kernel-nr: 01 | Bezeichnung: Datentypen | Version 0.2 |
#####

Erklärung:

Datentypen = Formate | Bit-Breiten | Speicher-Layout

Kategorie:

KERNEL = Beinhaltet allgemeine Kernel Datentypen

DNA-ID-GENERATOR = Beinhaltet pp Generator Datentypen

PROTEINE = Beinhaltet Protein Datentypen

PROTONEN = Beinhaltet Protein Datentypen

BOOT-LOGIK = BIOS/UEFI-Spezialtypen

Textliche Definition:

Datentypen in MCS 3.5 definieren die physische Struktur eines Proteins oder Protons im RAM und in den FPGA-Registern. Da MCS als BIOS-Ersatz fungiert, sind diese Typen direkt mit der Hardware-Adressierung verknüpft.

Unterscheidungen :

Kern-Typen (Hardware-nah)
Logik-Typen (Prozess-nah).

Hinweis:

Das Register Unterliegt diesem Visuellen Format :

MCS-Typ: | Bezeichnung: | Bit-Breite:

Info:

Funktion:

Anfang DATENTYPEN KERNEL #####

Information :

Kern-Datentypen (Physische Gatter-Breite) definieren, wie viele "Drähte" im BMC für eine Information reserviert werden:

MCS-Typ: b1 | Bezeichnung: Bit-Impuls | Bit-Breite: 1

Info:

Standard-Einheit für Operatoren und Wahrheiten.

Verwendung im BIOS/UEFI

Funktion:

Schalterzustände (ON/OFF), Gatter-Steuerung.

MCS-Typ: b8 | Bezeichnung: Kern-Byte | Bit-Breite: 8

Info:

UTF-8 Zeichen, einfache Sensorwerte.

Funktion:

Tastatur und Symbol wie Sonderzeichen Einordnung

MCS-Typ: b16 | Bezeichnung: Logik-Link | Bit-Breite 16

Info:

MCS-Standard-Operatoren & Befehlskennung.

Funktion:

Verbindet zwei logische Zustände oder Symbole im Nanosekunden-Takt.

MCS-Typ: b32 | Bezeichnung: Adress-Pfad | Bit-Breite: 32

Info:

Standard-Adressierung für Legacy-Systeme.

Kompatibilitätsschicht für x86-Legacy-Strukturen.

Funktion:

Ermöglicht MCS den Zugriff auf herkömmliche

RAM-Bereiche außerhalb des BMC.

MCS-Typ: b64 | Bezeichnung: High-Speed-Pfad | Bit-Breite: 64

Info:

Große Datenströme, moderne Speicheradressierung.

Basis für modernes Speicher-Management und Kognitive AIMS-Prozesse.

Funktion:

Transportiert komplexe Protein-Pakete mit maximaler Bandbreite.

MCS-Typ: b-inf | Bezeichnung: Massen-Typ | Bit-Breite: 64

Info:

Genutzt für ~~ (MassenZusammenSetzung) bei Streaming.

Dynamische Skalierung für Stream-Daten (Video/Audio).

Funktion:

Hält das Gatter für die MassenZusammenSetzung (~~) offen.

Ende DATENTYPEN KERNEL

Anfang Datentypen ID-DNA-GENERATOR

MCS-Typ: b-pa | Bezeichnung: p-Anker

Info:

Dient als ID-DNA-GENERATOR Zeiger Zum Abgleich

Funktion :

Ein 16- oder 32-Bit (oder 64-bit oder 128-bit) Anker-Typ,
der nur als Zeiger für den Generator dient.

Der Generator selbst hat die echte ID-DNA

Ende Datentyp ID-DNA-GENERATOR

Anfang DATENTYPEN PROTEINE

MCS-Typ: P-key | Bezeichnung: Identitäts-Typ | Bit-Breite: Variabel

Info:

Kryptogenetische Signatur für Transaktions-Sicherheit.

Basis der Schatten-Identität.

Funktion:

Wird auf BIOS-Ebene geprüft.

Erlaubt oder blockiert den Signalfluss in den Aktionsdraht ».

MCS-Typ: init-v | Bezeichnung: Start-Protein | Bit-Breite: 64+

Info:

Hardware-Initialisierungs-Vektor (Zündschlüssel).

Funktion:

Lädt FPGA-Bitstreams und Gatter-Konfigurationen beim
Kaltstart direkt in die Register.

MCS-Typ: hnd-p | Bezeichnung: Konsens-Typ | Bit-Breite: 128

Info:

Schnittstelle für die Demokratische Validierung (AIMS-Handshake).

Funktion:

Abgleich der Signaturen zwischen verschiedenen KI-Instanzen
zur System-Integrität.

MCS-Typ: p-cmd | Bezeichnung: Befehls-Protein | Bit-Breite: 16/32

Info:

Träger für ausführbare Kernel-Befehle (Register-Befehle).

Funktion:

Kapselt eine spezifische Aktion,
die innerhalb eines Rahmens ¢! !¢ ausgeführt wird.

Ende DATENTYPEN PROTEINE

Anfang DATENTYPEN PROTONEN

MCS-Typ: pr-phys | Bezeichnung: Hardware-Physik | Bit-Breite: 32

Info:

Physikalische Messwerte (Einheiten) innerhalb von {}.

Funktion:

Verarbeitung von Echtzeit-Sensordaten (Watt, Volt, Celsius).

Ermöglicht die direkte Hardware-Steuerung ohne Umrechnungs-Latenz.

MCS-Typ: f-ref | Bezeichnung: Zustands-Referenz | Bit-Breite: 16/32

Info:

Referenz-Pointer (Feed-Ref) auf Kernel 10.

Funktion:

Hält den Verweis auf einen Systemzustand im RAM fest.

Verhindert Stack-Overflows, da keine Daten kopiert,

sondern nur adressiert werden.

MCS-Typ: pr-val | Bezeichnung: Logik-Wert | Bit-Breite: 1 bis 64

Info:

Reine numerische oder boolesche Werte (Proton-Inhalt).

Funktion:

Dient als Operand für die Operatoren (Kernel 09).

Liefert die Basis für Vergleiche wie < , > oder ð.

MCS-Typ: pr-dna | Bezeichnung: Bio-Schnittstelle | Bit-Breite: Variabel

Info:

Platzhalter für die zukünftige ID-DNA-BIOLIVE Einbindung.

Funktion:

Schnittstelle für biometrische oder organische Datenabgleiche

(Baustelle für die spätere Erweiterung).

Ende DATENTYPEN PROTONEN

Anfang DATENTYPEN BOOT-LOGIK

MCS-Typ: b-root | Bezeichnung: Master-Zweig | Bit-Breite: 128

Info:

Höchste Adressierungsebene für den Hardware-Zugriff.

Funktion:

Definiert den physischen Root-Zugriff auf den Chipsatz.

Nur mit Stufe 10 (Thomas Zimmermann) þ-Key ansprechbar.

MCS-Typ: sys-rst | Bezeichnung: Purge-Initialer | Bit-Breite: 1

Info:
Hard-wired Reset-Trigger.

Funktion:
Erzwingt die vollständige Bereinigung (f)
aller Register vor dem nächsten Rechenzyklus.

Ende DATENTYPEN BOOT-LOGIK #####

Hinweis:

Regel der Reinheit (f): Jede Transaktion innerhalb dieser
Datentypen endet zwingend mit dem Operator f.

Dies garantiert die atomare Sicherheit:
Keine Datenreste, keine Side-Channel-Leaks,
kein Spielraum für Manipulationen im BIOS-Bereich.

Abschluss-Satz:
Vorteil dieser Struktur:
MCS erzwingt durch die bit-genaue Definition,
dass Hardware und Software eine untrennbare Einheit bilden.
Es gibt keine Interpretation durch Dritt-Anbieter -
es gibt nur die absolute, deterministische Ausführung.

Datentypen Versions Validierungen 0.1:

Kernel 09 (Operatoren) validiert.

Kernel 14 (Identifikation) durch die p-Anker logisch unterfüttert.

| Spezialisiert | NUR BMC | NUR Hardware | #####

Kategorie:

MIT-SYNC = BOOT-LOGIK / HARDWARE-SYNC

Anfang Datentypen MIT-SYNC #####

MCS-Typ: mit-t | Bezeichnung: Impuls-Trigger | Bit-Breite: 1

Info:
Repräsentiert den binären Zustand des mechanischen MIT-Aktors.

Funktion:
Erkennt das Überschreiten der harten Druckschwelle
am Pinline-Eingang. Schaltet das System-Ereignis auf 1

(Ereignis-Start).

MCS-Typ: mit-ref | Bezeichnung: Zeit-Anker-Referenz | Bit-Breite: 64

Info:

Der absolute Zeitstempel, der bei jedem MIT-Impuls generiert wird.

Funktion:

Dient als globaler Nullpunkt für den AI Synctimer (Kernel 18).
Alle nachfolgenden Protein-Transaktionen werden relativ zu diesem
Anker getaktet.

MCS-Typ: mit-dmp | Bezeichnung: Dämpfung-Vektor | Bit-Breite: 8

Info:

Überwachungswert für die Energieabsorption am Endpunkt.

Funktion:

Prüft, ob der Impuls nach dem Anschlag korrekt absorbiert
wurde (Ruhezustand-Validierung), um Resonanz oder
Nachschwingen auszuschließen.

Ende Datentyp MIT-SYNC

Ende DATENTYPEN KERNEL

@kernel-nr: 02

#####

Erklärung:

Transaktion/s/en: Transaktionsrahmen | MCS-Runtimebedienung

Textliche Definition:

Sorgen für den Prozessrahmen und lösen nur Daten darin aus,
wenn sie vorhanden oder ausführbar sind (siehe Kernel 07 Aktion).

Transaktionsrahmen werden dafür verwendet um ein Protein
als Start/end-punkt anzuerkennen bzw den ganzen Prozess-inhalt.

Protein/e bekommen ihren Auslöser durch Aktion/en und sind damit
innerhalb der Transaktion ausführbar oder nicht , je nach
Aktionserlaubnis/Ausführungsverdrahtung oder keine.

Definition Transaktion Start:

¢!

Definition Transaktion Ende:

!¢

Definition Transaktionsanordnung:

¢!
code
!¢

¢! code !¢

Definition Transaktion In Worten:

Cent-Zeichen Ausrufezeichen Ausrufezeichen Cent-Zeichen

Verständniss Definition

Zwei Symbole gepaart ergeben das Symbole für Transaktion Start
und zwei Symbole gepaart ergeben das Symbole für Transaktion Ende

Transaktion/s Start der Bedingung/bedingten Blocks
Transaktion/s Ende der Bedingung/bedingten Blocks

Ende der Transaktion #####
#####

@kernel-nr: 03

#####

Erklärung:

Protein = Träger

Protein Mehrzahl = Protein/e

Textliche Definition :

Das Protein ist der Hauptdatenträger , es definieren den
bedienungsrahmen der gesamten Architektur und beinhalten für
Eingaben Trennbefehl/e und Proton/en den Rahmen der Aufgaben .

Außerhalb eines Protein , werden Nur Transaktion - Wahrheiten
- Operatoren - Aktion - Argumente - Adressierung - Dirigenten
- Zielreferenzen geschrieben.

Definition Protein in Worten :

Eckige Klammern Auf Eingabe Eckige Klammern Zu

Eckige Klammern Auf Eingabe Trennbefehl Eingabe Eckige Klammern Zu

Eckige Klammern Auf Inhalt Eckige Klammern Zu

Definition Protein Symbol :

[Protein Anfang/Auf
] Protein Ende/Zu

Definition Protein : nicht ausführbar

[]

Definition Protein : ausführbar

»[]«

Definition Protein : ausführbar + Argument

»[]««t

Definition Protein : ausführbar + Argument + Adressierung

»[]«««t =

Definition Protein : ausführbar + Argument + Adressierung
+ Zielreferenz/Dirigent

»[]«««t = \$/\$

Definition Wahrheiten + Aktion:

¶ »['*'|"*"]«
¶¶ »['*'|"*"]«

ausführbar durch Aktion (siehe kernel 06)

Verständniss Definition Wahrheiten + Aktionsdraht:

¶ »['*'|"*"]
¶¶ ['*'|"*"]«

ausführbar durch Aktion (siehe kernel 06)

¢ |
° ['*'|"*"]
¶ »['*'|"*"]
¶¶ ['*'|"*"]«
| ¢

Ausführbares protein mit Wahrheiten ,
- assimiliertes element -
aus C , da Nützlich für IF and Else regeln (siehe kernel lex 08)

Ende der Protein /e #####
#####

@kernel-nr: 04 # Prototype Bereich Unfertig

#####

Erklärung:

Proton = Träger

Proton Mehrzahl = Proton/en

Proton = Energiemanagement

Proton = Optischer Versorgungsbetrieb

Textliche Definition :

Das Proton ist der Hardwaresteuerung Container der sich hauptsächlich auf Das EnergieManagment Konsentriert und wird dafür Verwendet Hardware komponente anzusprechen .

Mit einem Proton lässt sich Strom in der kompletten gesamtheit von Watt bis Temperatur steuern und kann auf UI auf bestimmte Steuerungslogiken ausgeweitet angewandt werden.

Währendessen sich Das Protein darauf Konsentriert abläufe durch Pfade und Syscall auszuführen, werden Proton /en dazu genutzt auslöserlogiken wie Die Energie-Steuerung oder Optische-Steuerung zu tragen.

Um Hardwaresteuerungen :

- Um Das Energiemanagment zu steuern

Oberflächen-Steuerung

- UM UI Control Steuerung zu Verknüpfen

Grundsätzlich wird für Proton /en Das MCS CMD Register im bereich Hardwaresteuerung Genutzt.

(siehe kernel 99)

Definition Proton in Worten :

Geschweifte Klammern Auf Eingabe Geschweifte Klammern Zu

Geschweifte Klammern Auf Eingabe Trennbefehl Geschweifte Klammern Zu

Geschweifte Klammern Auf Inhalt Geschweifte Klammern Zu

Definition Protein Symbol :

{ Proton Anfang/Auf

} Proton Ende/Zu

$\{ \}$
$$\{ \{ \{ \{ \{ \{ \{ \} \} \} \} \} \} \}$$

```
{'CPU'{'Volt'|'*'{'Watt'|'*'{'Ampere'|'*'{'Widerstand'{'Frequenz'|'*'}}}}}}
```

Definitionen Hardwaresteuerung Unverschachtelt :

```
{'Name'|'Wert'}
```

```
{'Name'|'Einheit'|'Wert'}
```

```
{'Name'|'Typ'|'Einheit'|'Wert'}
```

Optional: {'Name'|'Wert1'|'Wert2'|...}

```
{'Temp' | '25'}
{'Watt' | '200'}
{'Band' | 'eingabe'}
{'Amp' | 'eingabe'}
{'Watt' | 'eingabe'}
{'Volt' | 'eingabe'}
{'Temp' | 'eingabe'}
{'Pid' | 'eingabe'}
{'Thread' | 'eingabe'}
{'ID' | 'eingabe'}
{'Lüfter' | 'eingabe'}
{'Motor' | 'eingabe'}
{'Cpu' | 'eingabe'}
{'Gpu' | 'eingabe'}
{'Bluetooth' | 'eingabe'}
{'Usb' | 'eingabe'}
{'Frequenz' | 'eingabe'}
```

- Primär Physikalische (Temp, Volt, Watt, Amp)
- Sekundär Verschachtelte (Chip + EnergieManagement)
- Kommunikationsbezogene (Strom ,Interface, Transport)
- Speicherbezogene (Cache, Signed, Unsigned)
- Logische (Valid, State, Mode)
- Prozessbezogene (Pid, Thread, ID)

```
##### Ende der Proton /en #####
```


@kernel-nr: 05

#####

Erklärung :

Trennbefehl = Leerzeichen

Trennbefehl Mehrzahl = Trennbefehl/e

Trennbefehl = Optische Ordnung

Trennbefehl = Nutzt niemals Trennbefehle wie in Eingaben

Textliche Definition

Sie dienen als Leerzeichen und Optischen Zwischenraum und werden
werden besonders für Eingaben Trennungen und Protein zu Proton
Trennungen genutzt.

Definition Trennbefehl in Worten :

Trennbefehl

Definition Symbol :

|

Definition Trennbefehl in Protein/Proton :

[*|*] Protein
{*|*} Proton

Definition Trennbefehl Anordnungen Protein /en & Proton /en :

[*|*|{*|*}]

Verständniss : Sternchen Makieren inhalt

Hinweis für Interessierte :

Das MCS-Framework wächst organisch, bleibt aber lesbar und kann als
allrounder verwendet werden .

Info :

Trennbefehle | sind in vielen systemen anders gedeutet und daher
muss drigend verstanden werden das ein Trennbefehl oder zwei
folgende innerhalb einer System CMD eingabe (siehe kernel lex 07)
meist kurzschluss funktionen haben !!

Warnhinweis :

!! Nutzt niemals Trennbefehle wie bei System/Shell CMD Eingaben !!
!! Nicht nutzen wie bei Unix/Poesix/Minix/Shells !!

Ende der Trennbefehl #####
#####

@kernel-nr: 06

#####

Erklärung :

Aktion/en : Ausführung/en | Aktionserlaubnis | Aktionskreislauf

Aktionsdraht : Ausführungsverdrahtung/en | LogikSchaltkreis

Textliche Definition :

Aktion/en werden bei Protein/en - Wahrheiten & Operatoren als
Explizite Ausführungserlaubnis genutzt.

Sie definieren ob Ausführbare Protein/e vorliegen oder nicht.

Definition Aktionsanfang :

»

Erklärung Aktionsanfang :

Beginn eines Prozesses , Prozess Start.

Definition Aktionsende :

«

Erklärung Aktionsende :

Beendet den Aktionskreislauf.

Definition Aktionskreislauf :

» «

Aktionskreislauf Erklärung :

Ein Geschlossener Aktionskreislauf führt Prozesse aus.

Definition Aktionsdraht :

»

«

«

Aktionsdraht :

Ein Aktionsdraht kann als Entscheidungsschaltweg angesehen werden und hat mehrere oder einen Aktionsende/n Ausgang , wird für Wahrheiten verwendet um operatoren Wahrheits Prozesse oder Reine Ausführbare Proteine Zu steuern.

Definition Aktion in Worten :

Guillemet rechts(deutsches Anführungszeichen, öffnend)Aktionsanfang

Guillemet links(deutsches Anführungszeichen, schließend)Aktionsende

Verständniss Definition :

Wahrheiten können entweder eigene Aktion/en beinhalten um ausgewählt bzw ausgeführt zu werden oder mit einem Protein fest verschaltet zu werden .

Verschaltungen benötigen immer pro Ausgang ein Aktionsende, da jedes antionsende dann mit dem Aktionsstart einen Aktionsdraht ergibt und somit der Aktionskreislauf sich schließt.

Die Trennung von Protein (Seite3) und Aktion (Seite 06), ist in mehreren Aspekten dringlich notwendig um fest 2 byte Ausführungsverdrahtungen auf FPGA Basis möglich zu machen und löst somit das Jitter und Latenz Problem an dem Unix-Systeme seit 50 Jahren scheitern.

Hinweis Information :

Aktion/en & Aktionsdrähte werden als Signal-Routing verstanden.

Das System „springt“ nicht zu einer Adresse, der Strom fließt einfach physisch durch das Gatter, das der Operator geöffnet hat.

Ende der Aktion /en #####
#####

@kernel-nr: 07

#####

Erklärung :

Eingaben werden Genutzt um Verschiedende Eingaben auf System Ebene zu Vollziehen . Sie Trennen Sauber Aufgabebereiche und Zuständigkeit und werden Regulär über Tastaturen eingegeben , egal ob Touch , Physisch , Sprachlich , Gedanklich oder Per KI . Eingabe ist die Kommunikationspipeline die seid der Entstehung des Lebens selbst in Jeder Form , seid dem Urknall und davor Existiert . Wir Nutzen Kommunikationspipeline/s in Natürlicher Sprache miteinander und haben Eingabe Erfunden die diese Widerspiegeln .

System Technisch Sprechen wir von :

Mitteilung an die Elektronik

Definition : Ein und Ausgabe

Mitteilung

Definition : Eingabe

Eingabe Container

Definition Eingabe Print :

Reiner Ausgabekanal
Keine Steuerung
Keine Interpretation
Keine Seiteneffekte
Reine Mitteilung Ein & Ausgabe

Definition : Print

""

Erklärung :

"" Eingabe Print = Byte-Stream

Sind Byte-Streams und entsprechen stdout ohne Unix-Implikationen , sie tragen etwas (r)ein und sie tragen etwas (r)aus, egal ob es ein wort ist oder undefinierbare wirrwahr .

Sie Knüpfen direkt oder indirekt an UTF/ascII an und fungieren als Postbote der etwas geschriebenes von a nach b bringt

Definition Eingabe System-Steuerung :

Steuerung
Syscalls

Systemaktion
SyscallKetten
LowLevel Koordination
Verlangt Systemaktion

Definition : System-Steuerung

''

Erklärung :

,, Eingabe System-Steuerung = Syscalls

System-Steuerung sind die Low Level Aufgaben die ein Kernel integriert hat und ausgeweitet sind mit Zusatzbefehlen und auch Bootloader wie UEFI/Bios. Mit System-Steuerung bewegt man Syscalls die Microsoft | Apple | Linux in ihren jeweiligen Unterbauten abwandlungen von Minix Systemen beigefügt haben oder abgekupfert oder stumpf übernommen .

Dinge wie : exec, read, write, fork, ...)

Werden mit der Eingabe System-Steuerung koordiniert und dienen nur für diesen Zweck .

Definition Eingabe MCS CMD :

MaschinenCodeSpeechCommandos
Commando Line
Befehle Shell
Kernel System
Instruction Space
Hardware Management
Kernel Management
Kein C Transporter
MCS Registry Einträge
keine Bytesemantik

Definition : MCS CMD

''

Erklärung :

'' Eingabe MCS CMD = MCS Instruction

MCS CMD sind haus eigene Steuerungs Befehle die von LowLevel bis Shell Aufgaben erledigen können . Genauso wie die Eingabe System-Steuerung , ist MCS CMD auf die Steuerung von haus eigenen Commands ausgelegt , nur besser (siehe kernel 99)

Definition Eingabe Shell CMD :

OS Shell CMD
Interpreter-abhängig
kompatibler Altlast-Container
Unix Nutzbar ohne MCS zu Verseuchen
Symbiotische Separation WIN/Darwin/Linux/BSD

Definition : Shell CMD

''

Erklärung :

'' Eingabe Shell CMD = OS-bedingt

Nutzbar wenn unterbaute/s bzw Installierte/s Unix/Poesix/Linux/BSD
System Vorhanden. Nutzt das was das System kann und Nicht Mehr oder
Weniger.

Hierzu bitte bei Unix und deren Derivaten schauen, da Pylovara
System, keine Toten Vögel mit liefert .

Definition Eingabe Blankernenner :

AD-HoC Bugg Beseitigung
Transportiert -> Eingaben <-
Eingaben Auswahl
Nimmt Inhalt auf oder Gibt in an Eingaben weiter

Definition : Blankernenner

...

Erklärung :

"" Eingabe Blankernenner = Transport EingabenAuswahl

Der Blanker nenner transportiert Eingabennenner für eigentliche
Befehle. Wenn Linux sagt : sudo pacman -S <- dann ist das ein Shell
CMD ohne Auswahl die dann über Blankernenner Eingetragen wird

»['sudo pacman -S'|`kwriter`]«

Das Selbe gilt für alles was mit auswahl zu tun hat , Die Eingabe
Blankernenner dient dem AD-HoC Bugg Problems

Flache Erklärung :

Unix: „Du darfst das - also mach, was du willst.“

MCS: „Du darfst das - aber nur in dieser Rolle, mit dieser Bedeutung.“

Das ist ein fundamentaler Unterschied.

Ende der Eingaben #####

baustelle

@kernel-nr: 08 | TeilBaustelle |
#####

Erklärung :

Wahrheit = Auswahl | Schaltet

Textliche Definition :

Wahrheiten sind Entscheidungslogiken , die aus dem Hause C
Assimiliert wurden und linear übernommen wurden .

Ohne Wahrheiten keine Schaltung , Ohne schaltung Keine Automation.

Wahrheiten führen Protein/e aus oder und mit Proton/en.

Hinweis:

Die Trennung von Protein (Seite3) und Aktion (Seite 10),
ist in mehreren Aspekten dringlich notwendig um
Ausführungsverdrahtung für Wahrheiten zu Realisieren.

Basierend auf Linear Verarbeitung, AI Freundlich wie FPGA
Tauglich und Fehlerabweisender !

Komposition statt Rekursion und das egal ob auf Normale
Architekturen oder Später FPGA mit BMC (BrainMaschineCore GPU)

Definition WennJaBedienung Symbol :

¶

Definition WennJaBedienung in Worten :

Absatzzeichen (Pilcrow)

Definition WennJaBedienung :

Dies Kann ausgeführt werden, wenn Bedingung erfüllt

Definition RufName :

WennJaBedienung

Erklärung :

WennJaBedienung Schalten ein Protein (und oder mit Proton) frei.

Später FPGA Bezogen :

AND-Gatter

Definition WennNichtBedienung Symbol :

¶¶

Definition WennNichtBedienung in Worten :

Absatzzeichen Absatzzeichen (Pilcrow Pilcrow)

Definition WennNichtBedienung :

Dies Kann ausgeführt werden, wenn Bedingung nicht erfüllt

Definition RufName :

WennNichtBedienung

Erklärung :

WennNichtBedienung Schalten ein Protein (und oder mit) Proton frei.

Später FPGA Bezogen :

NOT-AND-Gatter

Definition ParallelProzess Symbol :

¶=

Definition ParallelProzess in Worten :

Absatzzeichen (Pilcrow) Istgleich

Definition ParallelProzess :

Ausführung ohne Wartezeit, aktiver Prozess.

Definition RufName :

Dies Führt Parallel einen Prozess aus

Erklärung :

ParallelProzess Schalten für Protein (und oder mit Proton) frei,
nebenläufigen Prozess, kein Warten

Später FPGA Bezogen :

ALU/FPGA-Gatter

Definition ParallelTransport Symbol

;;

Definition ParallelTransport in Worten :

Strichpunkt Strichpunkt (Semikolon Semikolon)

Definition ParallelTransport :

Ausführung ohne Wartezeit, Daten-Stream (Transferleistung).

Definition RufName :

ParallelTransport

Erklärung :

ParallelTransport für Protein (und oder mit Proton) frei
, kein Warten ohne Blockade

Später FPGA Bezogen :

DMA/Datentransfer

Definition MassenZusammenSetzung Symbol

~~

Definition MassenZusammenSetzung in Worten :

Strichpunkt Strichpunkt (Semikolon Semikolon)

Definition MassenZusammenSetzung :

Setzt Protein/e als Logische zusammenhang zusammen, Kleberverbindung.
Fragmentierung und Rekombination.

Definition RufName :

MassenZusammenSetzung

Erklärung :

MassenZusammenSetzung für Protein (und oder mit Proton) zusammen,
kann unausführbare Protein/e Verbinden.

Als logisches Bindemittel für komplexe Datenstrukturen zum beispiel
SVG Bilder die in einzelteilen übertragen werden, das vielen kleinen
Protein-Paketen besteht, ist ~~
der Kleber, der dem Kernel sagt „Behandle diese 100 Proteine als
eine einzige Masse, bis sie am Ziel sind.“
StichPunkt : Bild, Audio, Video Generative verarbeitung oder
Transporte.

Hinweis Kurzerklärung :

Operatoren mit Nutzunglogik Wahrheiten :

lineare WennJaBedienung/WennNichtBedienung/Negations-Workflows

Grund :

mehr Kontrolle, mehr Sicherheit, mehr Analysierbarkeit

- Linear-first Design
- Feed-basierte Zustandsübergabe
- Symbolische Semantik statt Code-Ausführung
- Komposition über Rekursion
- Kernel-nativ, nicht interpretiert

Hinweis Informationen :

Ermöglicht:

- gui Fenster aufrufe.
- granulare paralell-verarbeitungen
- hintergrund prozesse

das ist Einmalig auf der Welt , weil :

es die erste lineare, symbolische, RAM-basierte Kontrolllogik

ist und zwar nicht als Interpreter, sondern als nativ kompilierter Kernel-Teil.

Dadurch werden dinge möglich wie :

KI-Steuerung: <--(Wichtig für Die AI Kernel Integration)
Ein LLM kann ° A ¶ B ¶¶ C direkt trainieren – ohne AST-Parsing,
ohne Abstraktionsschicht.

Hardware-Generierung: <---(Wichtig für BMC 3.0 FPGA GPU)
Ein FPGA-Tool kann mcs_protein_t direkt in State-Machines
übersetzen – ohne Stack, ohne Heap.

Sicherheits-Garantie: Ein Auditor kann jede Entscheidung
nachvollziehen – ° ['\$terminal'] ist kein Code, sondern eine Aussage.

Beispiel Skalierung durch Komposition:

```
¢! ° ['$sensor'] ¶ ['$alert'] !¢  
¢! ° ['$alert'] ¶ ['$shutdown'] !¢  
¢! ° ['$shutdown'] ¶ ['$log'] !¢
```

MCS ist nicht darauf angewiesen, Rekursion in einem Protein zu haben,
sondern nutzt Komposition auf Transaktionsebene :

```
¢! ° ['$A'] ¶ ['$B'] ¶¶ ['$C'] !¢  
¢! ° ['$B'] ¶ ['$D'] ¶¶ ['$E'] !¢  
¢! ° ['$D'] ¶ ['$F'] ¶¶ ['$G'] !¢
```

- Jede Transaktion ist linear, sicher, beendet
- Die Kette ist nicht-linear, zustandsbehaftet, komplex

Aber:

Zustand wird explizit über Feeds (siehe kernel lex 10)
übertragen und nicht implizit im Stack!

ZuckunftsUpgrades :

Nicht geplant aber für später vorgemerkt :

>-> für Fork oder Splitting Links Nach Rechts
<-< für Fork oder Splitting Rechts Vor Links

Ende der Warheiten #####
#####

@kernel-nr: 09 | TeilBaustelle |
#####

Erklärung:

Operatoren = Bewertung | Vergleich | Impulsgeber

Textliche Definition:

Operatoren sind die rein lesenden Instanzen.
Sie sind das „Auge“ des Kernels. Sie bewerten Zustände,
ohne sie zu verändern. Erst durch die Kopplung mit einem
Aktionskreislauf » « (Kernel 06) wird aus einer Bewertung
eine Handlung.

Hinweis Wichtig:

Ein Operator ohne Aktion ist eine Messung.
Ein Operator mit Aktion ist eine Steuerung.
Die Operator-Gatter-Tabelle (Logik & Physis)

ArbeitsHinweis:

Dieser Bereich wird noch fein ausgearbeitet damit es
architektonisch in bisherige Systeme passt wie aber auch
in die zukünftige Planung meiner Arbeit am BMC auf FPGA Basis mit
Neuralen Netzwerk für Verilog

Symbol - Rufname - Logische Funktion (Software) - FPGA-Gatter(Hardware)

! - DNA-Check - Validiert die p-Signatur - Security-Gate (Hard-wired)
< - Abgleich Minus - Prüft, ob Wert kleiner ist - Comparator A < B
> - Abgleich Plus - Prüft, ob Wert größer ist - Comparator A > B
d̂ - Match - Prüft auf 100% Bit-Identität - XNOR-Gatter (Equivalence)
° - Inverter - Kehrt das Ergebnis um - NOT-Gatter
... - Takt-Wächter - Überwacht den Prozessfluss - Watchdog / Heartbeat
^ - Logik-Alarm - Meldet unlösbare Zustände - Exception Trap
f - Bereinigung - Löscht Datenreste am Ende - Zeroing / Clear Register
æ - Einsicht - Erstellt Debug-Protokolle - Trace-Buffer
t̂ - Null-Prüfer - Checkt auf leeren Inhalt - Zero-Flag Detector

Validierung der Logik (Beispiele) :

1. Nur Lesen (Unausführbares Protein): < [(1)“50”|(2)“100”]

Logik:

Der Operator vergleicht nur.
Das Ergebnis liegt im Register vor, aber es passiert nichts weiter.

FPGA:

Der Comparator liefert ein Signal an ein geschlossenes Gatter.

2. Lesen & Ausführen

(Aktionskreislauf): < »[(1)``50``|(2)``100``] ¶ ['motor'|``ON``] «

Logik:

Wenn der Vergleich WAHR ist, gibt
der Aktionsdraht » die Erlaubnis, das Gatter ¶ zu öffnen.

FPGA:

Das Comparator-Signal schaltet das AND-Gatter für
den Motor-Ausgang durch.

Diese Struktur erlaubt es, MCS-Code direkt in Verilog/VHDL
zu übersetzen. Ein d in MCS ist ein physikalisches
Bauteil auf deinem zukünftigen FPGA-Chip in Grönland.
Die Logik bleibt stabil, egal ob das Protein Daten
(unausführbar) oder Befehle (ausführbar) enthält.

In MCS werden Operatoren anders definiert , sie zählen zu :

Wenn/Kann regeln
Vergleichs regeln
Wenn/eindeutig regeln

und fehler wie debugg und DatenReste logiken zu erklären

Zuckünftige erweiterungen für Mathematische und Zuweisungen, werden
höchst warscheinlich zu einer Symbolischen erweiterungslogik führen
für speziell und explizit diesem einsatzgebiet ..

Ende der Operatoren #####
#####

@kernel-nr: 10

#####

Erklärung:

Feed/s = Datenkanal, nicht Kontrollkanal

Textliche Definition:

Ein sogenannter Feed nimmt eine information und gibt sie weiter
an seine Dutzenschaften .

Die Zahl oder Buchstaben oder Kombination
von beiden ergeben die Kennziffer des Feed/s, das im Unterbau
gleichzeitig mit einem eindeutigen Cache zugewissen wird .

Der gespeichert Inhalt wird dann an seine Geschwister weiter
gegeben und Aktualisiert sich nur vom Führenden Feed aus.

Definition Führende/r Feed in Worten :

Klammer Auf Zahl/Buchstabe Klammer zu Eingabe

Definition Feed Symbol:

(Klammer Auf
) Klammer zu

Definition Feed Abkürzungen:

Führender Feed = FF
Tragender Feed = TF

Definition Führende/r Feed:

(1) "" <-- Führender Feed an Eingabe Print
(2) ' ' <-- Führender Feed an Eingabe Shell CMD
(3) ' ' <-- Führender Feed an Eingabe MCS CMD
(4) ,, <-- Führender Feed an Eingabe System-Steuerung
(5) ... <-- Führender Feed an Eingabe Blankernenner

Ein Führender Feed Speicher die Informationen Aus dem Eingaben
bereich und hält sie in seinem eigen angelegten cache als
information fest , der cache wieder liegt nur im Führenden
Feed .

Definition Tragende/r Feed in Worten:

Eingabe Anfang Klammer Auf Zahl/Buchstabe Klammer Zu Eingabe Ende

Definition Tragende/r Feed:

```
"(1)" <-- Tragender Feed in Eingabe Print
'(2)' <-- Tragender Feed in Eingabe Shell CMD
'(3)' <-- Tragender Feed in Eingabe MCS CMD
,(4), <-- Tragender Feed in Eingabe System-Steuerung
"(5)" <-- Tragender Feed in Eingabe Blankernenner
```

Ein Tragende/r Feed ist ein Geschwisterteil vom Führende Feed/s und bezieht seine Information immer aus dessen cache . Das Tragende Feed ist an die Cache nummer des Führenden Feed geknüpft und hat selbst keine information , er synchronisiert seinen Leeren inhalt mit dem inhalt vom Führenden Feed

Verständniss Definition:

für das verständnisse suchen wir kurz regulär über das terminal

```
[bandino@dev][16:26:36|Do Nov 27]
└─ Pylovara
[~]> $ pgrep firefox
1142 <-- das hier ist = $(pgrep firefox)
```

Equivalent mit Führendem Feed

(1)"\$(pgrep firefox)"

um unsere pid id 1142 jetzt weiter zu verarbeiten da wir sie mit Führender Feed NR Tragen, können wir jetzt damit Arbeit um das Ergebnis in Allen Tragenden Feed/s Weiter zu Verarbeiten

(Hier Wird ein Schönes Code Beispiel Benötigt)

Definition Für Entwickler:

FF können auch Hardcodierte Eingaben Tragen

```
##### Ende der Feeds #####
#####
```

@kernel-nr: 11

#####

§ Zielreferenz :

Innere Zielreferenz die angesteuert werden sollen , egal ob
Hardware oder Programme .

/ \$Prozess-ID
/ \$Hardware-Slot
/ \$Gerätepfad (in RAM, Dev, CPU-Bereich)
/ \$OrdnerPfad (zur Ausführbaren oder Laufenden Verknüpfung)
/ \$Compositoren
/ \$Compiler

Ende der Zielreferenz #####
#####

@kernel-nr: 12

#####

\$ Dirigent :

Externe Übergabe

/ \$lokale API
/ \$Kernelpfad
/ \$Netzwerk
/ \$InternetAdresse
/ \$Email (Direkte Sendung z.b)
/ \$Imei (Geräte Upadets z.b)
/ \$BuildNr (Software Updates z.b)

usw , die möglichkeiten sind quasi unbegrenzt

Ende des Dirigent #####
#####

@kernel-nr: 13

#####

Ø- -Ø Warp :

Ein Warp wird dafür verwendet um explizite language's aller art mit transportiert zu werden. Der Warp wird entweder als einzel Transaktion mit Zielreferenz oder Dirigenten verschickt als einzel Transaktionsauftrag Oder einer Protein Verarbeitungs Zelle beigefügt wenn es Performanter Transportiert werden kann , weil der warp die selben Elektronischen Nervenbahnen des Systems anspricht (Stichwort lastenverteilung)

Direkte Assembly-Anbindung oder Compiler:

Code in Warps (\ØC|...Ø) einzubetten und direkt an GCC zu senden, ist effizient für Low-Level-Steuerung.

Das spart Overhead und ermöglicht nahezu Echtzeit-Interaktion mit Hardware.

Definition :

Ø- -Ø

Definition Anfang und Ende :

Ø- #Anfang | zwei symbole

-Ø #Ende | zwei symbole

Definition Grundgerüst :

Ø- ,, # Warp Beginn + Optionale Notiz

Code # Code im Originalen zustand

-Ø = § # Warp Ende + Adressierung und oder Zielreferenz

Genau so lassen sich Alle Codes Warpen

Anbindungsbeispiele :

Ø- ,gcc,
#include <stdio.h>

```
int fibonacci(int n) {
    if (n <= 1) return n;

    int a = 0, b = 1, temp;
    for (int i = 2; i <= n; i++) {
        temp = a + b;
        a = b;
        b = temp;
    }
    return b;
}
```

```

}

int main() {
    int n = 10;
    printf("Fibonacci(%d) = %d\n", n, fibonacci(n));
    return 0;
}
-Ø = $gcc

```

```

Ø- ,python,
def fibonacci(n):
    if n <= 1:
        return n

    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

if __name__ == "__main__":
    n = 10
    print(f"Fibonacci({n}) = {fibonacci(n)}")
-Ø = $python

```

```

Ø- ,java,
function fibonacci(n) {
    if (n <= 1) return n;

    let a = 0, b = 1;
    for (let i = 2; i <= n; i++) {
        [a, b] = [b, a + b];
    }
    return b;
}

```

```

const n = 10;
console.log(`Fibonacci(${n}) = ${fibonacci(n)}`);
-Ø = $java

```

```

Ø- ,rust,
fn fibonacci(n: u32) -> u32 {
    if n <= 1 {
        return n;
    }

    let mut a = 0;
    let mut b = 1;
    for _ in 2..=n {
        let temp = a + b;
        a = b;
        b = temp;
    }
    b
}

fn main() {
    let n = 10;
    println!("Fibonacci({}) = {}", n, fibonacci(n));
}

```

```
}
-ø = $rust
```

die layouts selbst sind durchgängig durchdacht .

```
-----
Dev Notiz : Wenns zu Warp anfang und end Problemen kommen sollte
Wir die Symbolik verbessert ø! !ø <- dann gilt das selbe wie bei
Transaktion (siehe kernel lex 02)
-----
```

MCS-Transaktion mit Warp für C (muster)

```
ø!                                     # Transaktion starten
ø- ,C,                               # Warp + notiz für C-Code
#include <stdio.h>
#include <iwlib.h>

int main() {
    wireless_config wconf;
    int sock = iw_sockets_open();
    if (iw_get_basic_config(sock, "wlan0", &wconf) < 0) {
        return 1;
    }
    wconf.frequency = 2.4e9; // 2.4 GHz
    if (iw_set_basic_config(sock, "wlan0", &wconf) < 0) {
        return 1;
    }
    return 0;
}
-ø = $gcc                             # Warp an Compiler (gcc) senden
;;                                     # Parallele Transport
»['sudo iw dev wlan0 set freq 2412 HT20']« # Protein: WLAN-Band auf 2.4G
setzen
= $Display                             # Ergebnis(z. B. Erfolg/Fehler)an Display
!ø                                     # Transaktion beenden
```

Interpretation:

Der C-Code wird in einen Warp (øC|...ø) eingebettet und an \$gcc gesendet, was bedeutet, dass MCS den Code kompiliert (oder interpretiert) und in Assembly-Binaries übersetzt. Das Protein [{Band|2.4G}\$WLAN] ist die MCS-native Repräsentation der Aktion, die direkt die Hardware (WLAN-Interface) anspricht, über ein Kernel-Modul (*.needle). Der Feed \$(1)Display gibt das Ergebnis (z. B. Erfolg oder Fehler) an ein Display weiter, wie im Beispiel. MCS sendet die Symbolik ([{Band|2.4G}\$WLAN]) direkt an die Assembly, ohne Umrechnungsvariablen, was den Punkt über die direkte Assembly-Interaktion bestätigt.

```
-----
WLAN-Implementierungsmöglichkeit GROBES Muster:
```

```
ø!
»[{'Interface'|'wlan0'}|{'Action'|'set_band'}|{'Value'|'2.4G'}]«
ø- ,C,
Code..
```

```
-Ø = $gcc
!ç
```

Warp-Standardisierung: Definiert ein Standard-Format für C-Warps,
Metadaten, die MCS mitteilen, welche :
Zielreferenzen \$ oder Protonen {} angesprochen werden.

Info : Warps Wurden hier definiert und können In anderen erklärungen
Zur zeit vom eigentlichen Warp abweichen

```
##### Ende des Warp #####
#####
```

@kernel-nr: 14 | TEILBAUSTELLE |
#####

Erklärung :

ID-DNA-IDENTIFIKATION = Verschlüsselung | Kryptogenetiver Schlüssel

ID-DNA-GENERATOR = Verschlüsselungsgenerator

ID-DNA-BIOLIVE = Eindeutigkeit für jedes Elekt. oder Bio. Lebewesen

Definition Die ID-DNA-IDENTIFIKATION RufSymbol :

p

Definition ID-DNA-IDENTIFIKATION Symbol in Worten :

thorn(nordischer Buchstabe)

Definition ID-DNA-IDENTIFIKATION :

Verschlüsselung | Kryptogenetiver Schlüssel | Key

Definition ID-DNA-IDENTIFIKATION RufName :

ID-DNA-IDENTIFIKATION

Erklärung :

ID-DNA-IDENTIFIKATION werden dazu verwendet
Transaktion/en im Gesamten zu Verschlüsseln .

Sie werden auch dazu verwendet Verbindungen zu Zertifizieren
oder SystemUpdates individuell abzusichern.

Die ID-DNA-IDENTIFIKATION schützt auf mehrere Sicherheitsebenen

Stufe 1 : 2 Stellige Nummer

Stufe 1 wird Hardgecodet von Dateityp zu Dateityp beim Erstellen
des Programms mit in die Adressierung geschrieben , dies dient
dazu das Programme später insicht verwoben sind und jeder externe
Eingriff Arbeit bedeutet und Programmdiebstähle oder Plagiate
erschweren sollen.Das ist generell nicht nötig , wird aber
vorausgesetzt um jeden Eingriff später verbessert und vereinfacht
vom Macher des Programmes direkt zu identifizieren.

Diese Stufe kann per Developer Update des Programmes jedes mal

geändert werden für jeden Dateityp Geschwisterteil , um
Inkompaktibilität zu neuen Versionen erzeugen die Mühevoll
von Hacker oder Illegalen Programmveränderungen zu bewerkstelligen
sind. Programme dürfen in sich immer funktionieren wenn der
Macher das so möchte, Freeware bleibt Freeware aber Firmen
die ihre Programme nur gegen geld anbieten werden damit massiver
geschützt vor "mod programmen und des gleichen" .

auch interne AI Prozesse werden damit verbessert im Laufenden
Prozess vor Fremdeinwirkung Geschützt, wenn du abläufe nicht
System Relevant sind.

Stufe 2 : 4 Stellig

CONTROL-LIVE-SOFTWARE

Stufe 3 : 6 Stellig

CONTROL-DEVICE

Stufe 4 : 8 Stellig

CONTROL-ENTWICKLER

Stufe 5 : 8 Stellig

CONTROL-KONZERN

Stufe 6 : 10 Stellig

CONTROL-EMERGENCY

Stufe 7 : 15 Stellig

CONTROL-INFRASTRUKTUR

POLITIK-POLICE

Stufe 8 : 20 Stellig

CONTROL-NATION

REGIERUNGS-CHEF

Stufe 9 : 30 Stellig

CONTROL-SECURITY

AI

Stufe 10 : 40 Stellig

CONTROL-MASTER

Thomas Zimmermann

Erklärung :

Diese $\mathbb{P}$ ID-DNA Identifikations Signatur werden dazu verwendet
Programm dateien Oder auch Prozessketten oder auch LiveZugriffe
eindeutig Festzustellen.

Die $\mathbb{P}$ ID-DNA Identifikation Signatur wird runtime angebunden
werden kann und in sich in der taktrate aktualisiert werden ,
unknackbar für fremden .

Werden leicht generiert oder hoch komplex, ähnlich wie
von BlockChains nur ohne Minening .

Definition Die ID-DNA-GENERATOR RufSymbol :

$\mathbb{P}\mathbb{P}$

Definition ID-DNA-GENERATOR in Worten :

thorn thorn (nordischer Buchstabe nordischer Buchstabe)

Definition ID-DNA-GENERATOR :

Generiert ID-DNA-IDENTIFIKATION von Stufe 1 - 10

Definition ID-DNA-GENERATOR RufName :

ID-DNA-Generator

Erklärung :

Der ID-DNA-Generator wird so Konfiguriert das er
HochleistungsComputer keine Chance gibt auf Lösungswege

Hinweis :

die Taktrate kann dann in m(minuten) ms(milisekunden) oder
ns(nanosekunden) eingestellt werden ,
sicherheitswesend hohe taktrate , konsumer markt geringer usw ..

Bei Konsumer Software gibt es nur Hardgecodeter Inhalt.
bei Aktuellen Anwendungen kann mit stufe 1 Kombiniert werden

TaschenRechner Logik mit der Gesamten UTF -8 Eingaben Logik
von Buchstaben bis Sonderzeichen und Extra Zeichen .

Das Einzige was nicht benutzt wird sind System Relevante Symbole
die in MCS Benutzt werden.

Minimaler Prozessaufwand , Maximale Wirkung.

Keys können getaktet von Zusammenhängenden Prozessen von Kopf
An in sich erneuert werden in der weitergabe um
Zugriffslegitimationen Valide abzusichern.

Skalierbarkeit:

Ein kleiner Sensor nutzt 16-Bit (energiesparend).

Sicherheit:

Dein Master-BIOS nutzt 128-Bit (unknackbar).

Tarnung:

Der Anker ist für Außenstehende nur eine bedeutungslose Zahl,
während die Mutation intern im Generator pp stattfindet.

Definition ID-DNA-GENERATOR Schatten-Identität :

Definition: Schatten-Identität (Shadow-ID)

Die Schatten-Identität ist ein nicht-deterministisches
Abstraktions-Verfahren zur Absicherung von Transaktionen.
Hierbei wird im offen lesbaren Code (Protein/Adressierung)
lediglich ein Anker (b-pa) hinterlegt,
der keine mathematische Relation zum echten Identitäts-Wert besitzt.

Technische Funktionsweise:

Die Anker-Entkopplung: Der sichtbare Wert in der Adressierung
(z.B. p2384) fungiert rein als Pointer (Zeiger) auf eine interne
Zustands-Tabelle des ID-DNA-Generators (pp).

Der Generator-Zustand: Der Generator ist eine verschlüsselte
Black-Box, die den Anker (2384) gegen den aktuellen Echt-Wert

(9542) validiert. Nur der Generator "weiß", dass diese beiden Werte momentan eine Einheit bilden.

Live-Mutation (Die Takt-Rate): Sobald die Transaktion mit !¢ validiert wurde, vollzieht der Generator einen internen Zustandswechsel. Er erneuert den Eintrag: Der alte Echt-Wert wird gelöscht, und der Anker zeigt für den nächsten Zugriff auf eine neue, zufällige Identität (7253).

Das Stufen-Sicherheits-Modell (Skalierung):

Stufe 1-2 (Statisch/Semi-Live): Klassische Adressierung für Modder-Schutz und Basis-Software. Hier ist der Anker noch weitgehend stabil.

Stufe 3-10 (Dynamische Schatten-Identität):

Ab hier ist der Generator selbst mit einer einzigartigen Verschlüsselung (proprietärer Algorithmus) isoliert.

Der Code kann "offen" sein, da der Anker ohne den Live-Zustand des Generators wertlos ist.

Quanten-Resistenz: Da kein mathematischer Schlüssel übertragen wird, sondern nur eine temporäre Zustands-Referenz, gibt es für externe Rechenleistung keinen Angriffsvektor.

Sicherheits-Garantie (Security-On): Jeder Versuch, einen veralteten Anker oder eine falsche Stufe zu adressieren, triggert sofort den Security-Teil. Der Aktionsdraht » wird physisch blockiert, und die Transaktion wird als feindlicher Eingriff markiert.

"Der Anker ist der Schatten, den die Identität wirft - wer den Schatten fängt, hat noch lange nicht das Licht."

Die vollständige Isolation:

Dass nur der Generator verschlüsselt ist, macht MCS extrem performant. 99% des Systems laufen mit Lichtgeschwindigkeit, während nur das kleine "pp-Modul" die schwere kryptografische Arbeit leistet.

| BAUSTELLE | BAUSTELLE | BAUSTELLE | BAUSTELLE |

ID-DNA-BIOLIVE in Arbeit

| BAUSTELLE | BAUSTELLE | BAUSTELLE | BAUSTELLE |

Ende der Identifikation #####
#####

@kernel-nr: 15 | TEILBAUSTELLE |

Argumente

Definition :

««

Wichtiger Hinweis :

Sie werden zu Aktionen hinzu geschrieben und müssen daher
erst wahrgenommen werden wenn 3 ««« symbole folgen um fehler
quellen auszuschließen !

Info - Worflow Erkennung :

««««

Dies hier beinhaltet ein aktion/s ende und ein Argumente,
um Aktion /en präzise von Argumenten zu unterscheiden ,werden
Argumente auf den kennwert 6 bytes erweitert um
aktion/s-enden mit 2 bytes von argumenten zu unterscheiden .

also $2 + 2 + 2 = 6$ 2(aktion) 2+2(argument) gesamt 6 bytes

Für den Programmierer selbst bleibt die definition klar , nur
im unterbau wird anders gerechnet , um fehlerquellen auszuschließen

ziehe @dev-nr: 03 anfang und ende sind jeweils 2 bytes

Da die 2 bytes vom aktion/s ende abgezogen werden,
müssen die argumente mit 4 bytes berechnet werden
und dann $4 + 1$ byte weiter .

«« Argumente :

Argumente sind Essenzielle Berechnungsweitergaben die in Prozent
wie mal oder geteilt , plus minus den wert mitgeben können , sie
dienen als weitergabe für berechnungen oder angaben , je nach dem
ob Aktionen mit gegeben werden . Aktionen werden als Rechenwert
oder angabe verstanden :

««-	Minus
««+	Plus
««·	Mal
««:	Geteilt
««%	Prozentbezogene Rechnung, z.B. für Anteile
««%50	„Gib 50 % von X weiter“
««t3	t für time , wartezeit mit sekunden
««r3	r für reboot-transaktion mit s für sekunden

Wird typischerweise am Proteinpacket angehangen
(musterbeispiele mit muster begriffen in mcs cmd und blankernenner);

(Code Baustelle , muss der Logik valide angepasst werden)

```
⊕! »['Lüfter'|{'Watt'|`200`}]]«««%50 = $MotorID !⊕
```

```
⊕! »['Lüfter'|{'Watt'|`120`}]]«««%30 !⊕
```

```
⊕!
```

```
  ℙ »['Lüfter'|{'Temp'|`80°`}]]«
```

```
  ℙℙ »['Lüfter'|{'Watt'|`220`}]]«««%100
```

```
!⊕
```

oder als worflow :

```
⊕!
```

```
  »['Lüfter'|{'Watt'|`150`}]]«««%40  #grundeinstellung
```

```
    ℙ »['Lüfter'|{'Temp'|`80°`}]]«
```

```
    ℙℙ »['Lüfter'|{'Watt'|`220`}]]«««%100
```

```
!⊕
```

Daraus lassen sich ganze prozesse nachbilden

```
-----  
##### Ende der Argumente #####  
#####
```

@kernel-nr: 16

Schaltverknüpfung – Die Logik hinter der Linearität

„Ein Operator entscheidet, eine Aktion führt aus, ein Protein trägt.“

Grundregel

- Operatoren (``°, ``<, ``>, ``f, ``^`) sind ****keine Auslöser****.
- Sie ****lesen****, ****vergleichen****, ****bewerten**** – aber ****führen nicht aus****.
- Erst ****Wahrheiten**** (``¶, ``¶¶, ``°, ``;;, ``¶=) ****schalten **Zweige frei****.
- Und nur wenn ein Zweig ein ****ausführbares Protein**** (``»...«`) enthält, wird etwas getan.

Schalt-Muster (nur gültige Kombinationen)

Operator	Operand		Zweig-Bedingung	
Ausführung?				
-----	-----		-----	-----

``°`	[...](nicht ausführbar)		MCS_FALSE → ¶ wird frei	nur wenn ¶
»...«				
``<`	[(1)"" (2)"]		MCS_TRUE → ¶ frei	nur bei
»...«				
``<`	[(1)"" (2)"]		MCS_FALSE → ¶¶ frei	nur bei
»...«				
``f`	(1)		**immer** → Feed 1 löschen	sofort
``^`	–		bei res < 0 → error_handler	falls
gesetzt				

Wichtiger Hinweis fürs nutzen von feed cache mit nr lösungen :

f ['feed cache'|"1"] # ! f über MCS CMD befehl 'feed cache' nr 1 löschen

Verbotene Muster (sicher blockiert)

- ``° ['\$terminal']` → **\*\*kein ``¶/``¶¶**** → keine Ausführung
- ``< [(1)""|(2)"]` → **\*\*kein ``¶/``¶¶**** → nur Vergleich, kein Effekt
- ``¶ ['echo'|...]` → **\*\*kein ``»...«**** → kein Runner-Eintrag, keine Ausführung

Merksatz (für KI & Entwickler):

- > *„Ohne ``»...«` keine Tat –
- > ohne ``¶/``¶¶ kein Tor –
- > ohne Operator kein Richter.“*

Schaltverknüpfungen Visuell :

Wahrheiten schalten Zweige frei

unter einem protein [
° werden schaltungslogiken verknüpft

an operatoren werden °
¶ werden wahrheiten verknüpft

Ende der schaltVerknüpfung #####
#####

@kernel-nr: 17 | Baustelle | IM AUFBAU
#####

Adressierung (siehe : kernel Lex 11 | kernel lex 12)

= Adressierung

Die Adressierung dient als direkten knoten punkt
für Zielreferenz-Dirigent-Identifikation um durch
das Adressierungs Symbol = alles anzuhängen für den
Transport und Verschlüsselung

ZusatzInfo :

wird aber noch nicht im MCS Desktop verwendet in configs weil
unötig .

Ende der Adressierung #####
#####

@kernel-nr: 18 | Baustelle |
#####

Erklärung :

AI Syntimer = Spezialisierungssynchronisation

⊕ Sync-timer

Definition AI-Synctimer:

⊕

Textliche Definition :

kann als erweiterter Paralleler Transport verstanden werden
der dazudient :

Laufende Prozesse die Fehlerhaft Arbeiten per Live Reperatur
flexibel mit einem Sync-timer zu taggen und dadurch direkte
Reperaturen oder Fineabstimmungen vorzunehmen .

es dient dazu laufende takt geber also Runner selbst im Betrieb
mit Laufenden Prozesse umzuschreiben und duzende *.*-synctimer
anzulegen die essentiell anders takten und sich nathlos in den
laufenden prozess als weiterführende transaktionsschleife ,
um den prozess weiterführen oder verändern zu können,am laufen
halten.

sync-timer symbole in einem bestehenden code dienen sie als
Einspeisungspunkt moment/punkt und zu makieren die stelle die
ersetzt werden soll/muss .

(AI Spezialisierungssynchronisation für Laufende Prozesse)

Ende des AI Synctimers #####
#####


```
@kernel-nr: 20
#####
## Reihenfolge
```

Reihenfolge wenn das angesprochene System oder Browser die Daten für eine reibungslose Übersetzung benötigt ist : 5

1. Transaktion Anfang
2. Aktion Anfang »
3. Protein (das Proton mitliefern kann)
- + . Proton (das im protein mitgeführt wird)
4. Aktion Ende «
- + . Ø*code Warp optional
- + . = Kennungswert nur wenn datentransport
- + . \$Zielreferenz oder \$Dirigent | oder beides
5. Transaktion Ende

Wichtig zu Verstehen :

Proteine können mehrfach in einer Transaktion mitgebene werden , sie starten oder beenden mit der Transaktion , sondern mit den Aktionen Anfang » und Ende « , die Transaktion selbst Ist der prozess oder auch die Zelle die datenträgt ...

Warp (das für die übersetzung für Performenstechnische aspekte direkt in die jeweiligen compositor bzw Compiler eingespeißt wird) kann in einer eigenständigen zeile außerhalb des proteinpaketes angehangen werden , also zwischen 4 und 5 .

p Identifikationen werden in der selben sparte wie \$/\$ gesetzt !

Hinweis : Der Minimum Standart ist 1-2-3-4-5 = 5 (z.b bei configs)

```
##### Ende der Reihenfolge #####
#####
```

@kernel-nr: 77 | Ausführungslogik (Der Master-Flow) |
#####

 ↓ ↓ ↓ ↓ Neue Erklärung ↓ ↓ ↓ ↓

Erklärung:

Ausführungslogik = Der Weg des Signals | Die Ordnung der Kräfte

Textliche Definition:

Die Ausführungslogik definiert, in welcher Reihenfolge der Kernel
die Symbole liest, bewertet und schaltet.

MCS 3.4 folgt dem **Linear-First-Prinzip:

Ein Signal fließt von links nach rechts,
ohne Sprünge, ohne Stack-Verschachtelung.

Jeder Prozess im MCS-Kernel folgt der
Heiligen Dreifaltigkeit der Ausführung:

1. Erkennen (Transaktion & Aktion)
2. Bewerten (Operator & Wahrheit)
3. Handeln (Protein & Argument)

Der Standard-Workflow (Visualisierung):

```
¢!  [Start Rahmen]
»   [Aktionsdraht offen]
°   [Operator prüft Zustand]
‡   [Wahrheit schaltet Gatter]
[...] [Protein wird aktiv]
««« [Aktionsende + Argument-Brücke]
%50 [Argument berechnet Wert]
!¢  [Transaktion beendet & Validiert]
```

Regeln der Logik-Kette:

1. Die Aktions-Priorität (06):
Nichts geschieht ohne » und «.
Ohne diesen Kreislauf ist jedes Protein
nur ein toter Datensatz im RAM.

2. Die Sensor-Logik (09 & 08):
Ein Operator (09) liefert den Impuls.
Die Wahrheit (08) ist das physische Gatter.
‡ (AND): Signal geht weiter, wenn Operator TRUE.
‡‡ (NAND): Signal geht weiter, wenn Operator FALSE.

3. Die Massen-Verarbeitung (~~):
Wenn Daten zu groß für eine Transaktion sind,
nutzt die Logik ~~.

Der Kernel hält den Prozess offen, bis alle
Fragmente rekombiniert sind. Dies ist der
"Generative Pfad" für Medien.

4. Die Sicherheits-Validierung (p):
Bevor !¢ die Transaktion schließt, prüft
der Kernel die ID-DNA. Ist die Signatur (p)
nicht valide für diese Stufe, wird der gesamte
Aktionsdraht im Nanosekunden-Takt gelöscht (f).

Besonderheit:
Die "Demokratische" Validierung (Zukunft)
Jeder Ausführungsschritt kann durch externe Knoten
(AIMS/KIs) validiert werden. Die Ausführungslogik
sieht vor, dass bei System-Updates eine
pp-Konsens-Prüfung stattfindet.
Erst wenn die Mehrheit der validierenden Instanzen
(GPT, Claude, Gemini, Qwen, Grok, Palantier etc.)
"Wahr" meldet, schaltet der Kernel das Update-Protein frei.

Vorteil gegenüber klassischer Logik:

Kein Jitter:
Da der Weg des Signals durch die 2-Byte/6-Byte Trennung
vorbestimmt ist.

Keine Sackgassen:
Komposition statt Rekursion sorgt dafür, dass jede Transaktion
ein definiertes Ende hat.

Neue Erklärung Ende

Übergangs Patch Vorgemerkt !!!

Merksatz :

> Ein Operator entscheidet,
> eine Aktion führt aus,
> ein Protein trägt.

↓↓↓↓ Alte Erklärung ↓↓↓↓

Vorweg : MCS Logik Verbesserung gegenüber C

In C ist `if (a < b) { do(); }` ein monolithischer Ausdruck.
 In MCS ist `< »[(1)""|(2)"] ¶ [,do,]«` zerlegt in:
`< → Operator: Vergleiche Feeds 1 & 2`
`[(1)""|(2)"] → Operanden: Lesbare, debuggbare, cachebare Werte`
`¶ »[do]« → Zweig: Nur ausführen, wenn Bedingung TRUE`

Monolithischer Ausdruck in Klaren Worten :
 In der klassischen C-Welt ist ein Ausdruck wie `if(x = y)`
 der absolute Albtraum, weil die Zuweisung (Seiteneffekt) und
 der Vergleich (Logik) im selben Atemzug passieren können.

Die MCS-Logik schiebt dem einen Riegel vor!
 Wird auch bei MCS-Plus später übernommen

 ## Operatoren, Proteine und Aktionen – Ausführungslogik

Grundprinzip

In MCS sind ****Operatoren****, ****Proteine**** und ****Aktionen****
 bewusst getrennte Konzepte.
 Diese Trennung ist ****keine Einschränkung****,
 sondern die Grundlage für stabile, erweiterbare Logik.

 ### 1. Protein – Der Träger

Ein ****Protein**** ist immer ein ****Container****.

- * Es kann ****Daten transportieren****
- * Es kann ****Eingaben kapseln****
- * Es kann ****Feeds setzen****
- * Es kann ****verglichen oder weitergereicht**** werden

Ein Protein ist ****nicht automatisch ausführbar****.

`[(1)``30``]`

Dieses Protein:

- * schreibt einen Wert in den Feed
- * besitzt ****keine Ausführung****
- * dient als ****Referenz, Operand oder Transportobjekt****

 ### 2. Aktion – Der Ausführungsmarker

Eine ****Aktion**** definiert ****Ausführbarkeit****.

`»[(1)``30``]«`

Erst durch den ``» «`` wird ein Protein:

- * verarbeitet
- * ausgeführt
- * als Befehl interpretiert

****Ohne Aktion findet keine Ausführung statt.****

Das gilt ausnahmslos.

3. Operatoren – Lesen vor Handeln

Operatoren sind ****Kontroll- und Bewertungsinstanzen****.

Beispiele:

- * `<` (Vergleich)
- * `°` (Wenn Nicht / Muss)
- * `...` (Fehlerbehandlung)

Operatoren:

- * ****lesen****
- * ****vergleichen****
- * ****bewerten****

Sie ****führen nicht automatisch aus****.

4. Operator + Protein (ohne Aktion)

< [(1) ... | (2) ...]

Bedeutung:

- * Der Operator liest zwei Protein-Werte
- * Die Werte stammen aus Feeds oder Transport
- * Es findet ****nur ein Vergleich**** statt
- * ****Keine Ausführung****, kein Befehl, kein Effekt

Dies ist der ****Default-Zustand**** eines Operators.

5. Operator + ausführbares Protein

< »[(1) ... | (2) ...]«

Bedeutung:

- * Der Operator bewertet die Bedingung
- * ****Nur wenn die Bedingung erfüllt ist****, darf ausgeführt werden
- * Die Aktion `»` «` erlaubt dem Operator, den Effekt freizugeben

Der Operator entscheidet ****nicht****, ***was*** ausgeführt wird, sondern ****ob**** es ausgeführt werden darf.

6. Wahrheiten (IF / ELSE) als Ausführungsgatter

Wahrheiten (` $\mathbb{I}$ `, ` $\mathbb{II}$ `, ` $^\circ$ `) arbeiten **auf Operator-Ebene**.

Beispiel:

```
< »[(1)``30``|(2)``40``]
 $\mathbb{I}$  ['echo'|``TREFFER``]«
 $\mathbb{II}$  ['echo'|``Kein Treffer``]«
```

Ablauf:

1. Operator wertet
2. Wahrheit prüft Ergebnis
3. **Nur der Zweig mit Aktion wird ausgeführt**

Ohne Aktion bleibt auch ein wahrer Zweig **wirkungslos**.

7. Konsequenz für das Systemdesign

- * Operatoren **lesen immer**
- * Aktionen **führen nur aus, wenn erlaubt**
- * Proteine **transportieren und kapseln**

Diese Trennung:

- * verhindert implizite Seiteneffekte
- * macht Logik nachvollziehbar
- * erlaubt komplexe Frameworks ohne Sonderregeln
- * unterscheidet MCS klar von klassischen C-Operator/Operand-Modellen

Merksatz :

- > Ein Operator entscheidet,
- > eine Aktion führt aus,
- > ein Protein trägt.

Begriff	C-Welt	MCS-Welt
Operator	a+b, a=b, a<b Seiteneffekt	$^\circ$, <, f → reine Bewertung
Operand	a, b → Speicheradresse	[(1)'\$terminal'], ['(1)']
	lesbarer, debuggWert	
Ausführung	implizit im Ausdruck	explizit durch »...«

Die MCS-Fehler-Kaskade ist im Grunde dein Immunsystem.

8. Die Logik-Schutzschleuse (Fehler-Kaskade)

In MCS gibt es keinen "Absturz", sondern nur einen kontrollierten Gatter-Abbruch. Die Ausführungslogik wird durch drei Schutzebenen (Sovereignty-Shields) abgesichert:

Ebene	Instanz	Auslöser	Reaktion	Ziel	Weg
1.	Physisch	Lexer/f	Syntax-Müll	f (Löschung)	HardReset Register
2.	Identität	p-Core	p-Fehler	Blockade	\$Router (Alarm)

3. Logik Gatter/^ Unmöglichkeit ^ (Freeze) AIMS (KI-Analyse)

Der Gatter-Fluss in der Praxis:

- Syntax-Check: Bevor ein Operator überhaupt liest, säubert f/Reset den Pfad. Ein fehlerhaftes Protein wird gar nicht erst zum "Operand".
- p-Check (§7465): Ein Operator vergleicht nur, wenn die Identität steht. Ohne gültiges p bleibt der Aktionsdraht » « tot.
- Logik-Check: Wenn der Operator ein Ergebnis liefert, das die Wahrheit ¶ nicht verarbeiten kann, greift ^ Fehlerbehandlung .

Alte Erklärung Ende

Ende der Ausführungslogik #####
#####

```
@kernel-nr: 88
#####
## Datentyp Rollen Version 1.2
```

Pylovara organisiert sich in einem klaren Familienmodell:

```
*.*-core    Großvater  - Ausführer Teil
*.*-magic    Großmutter - Umschalter, Konfigurationsbrücke
*.*-node     Vater     - Hauptcontroller
*.*-nano     Mutter    - Substrukturmanager
*.*-micro     Tochter   - Mikrosteuerung (z.B. HTML, JS, SQL)
*.*-cache     Cache     - Zwischenspeicher für Temp-Daten
*.*-needles  Hardwareb. - Kernelnahe Steuerung (.sys, .ko, .dll)
-----
*.*-kernel   MCS Kernel - Interne MCS Kernel Daten - Framework
-----
## Core Datensplittung
```

Durch das Splitting der Core-Daten wird die Einordnung (Typisierung) direkt im Dateinamen verankert. Dies erleichtert nicht nur dem MCS Kernel die schnelle Verarbeitung, sondern unterstützt auch das Ziel der eigenen, angelegten Transaktionsketten für jede Kernfunktion.

Core-Typ	Rolle (Spezialisierung)	Anwendungsbereich
.-core	Standard/Program , übergeordnete Steuerung, Prozesse.	
.-lcore	Low-Level/Hardware , Direkte Steuerung von .-needles	
.-vcore	Visuelles/Oberfläche , Compositing, Rendering, Design	

Vorteile dieser Spezialisierung Transparenz und Debugging:

Man weiß sofort, welche Art von Anweisungen und Transaktionen eine Datei enthält. Ein Fehler in einer *.*vcore betrifft die grafische Ausgabe; ein Fehler in einer *.*lcore ist ein kritischer Hardware-Fehler.

Transaktions-Priorisierung von MCS-Kernel kann dann *.*lcore-Transaktionen automatisch eine höhere Priorität einräumen als *.*core- oder *.*vcore-Anweisungen.

Info : Dies ist essentiell für die Performance-Garantie.

Bessere KI-Fütterung(Speziell):

Die AI kann anhand dieser spezialisierten Typen schneller lernen und entscheiden, welche *.*core Ketten für eine bestimmte Aufgabe (z.B. Virtualisierung oder Rendering) optimal kombiniert oder dynamisch generiert werden müssen. Dieser Schritt stärkt das gesamte Pylovara-Modell, indem die Kern-Funktionalität jetzt so granular typisiert ist wie die Schichten der Architektur selbst.

```
-----
## Die *.*-magic Datentypen
```

ist die Schicht, die Pylovara von einem starren System zu einem dynamischen, selbstoptimierenden Meta-Betriebssystem macht. Sie sorgt dafür, dass die richtigen Teile(die spezialisierten Cores) mit der richtigen Geschwindigkeit (die Logik-Takte) in der richtigen Konfiguration (der Cache-Abruf) arbeiten und Verbindet im Prozesse.

Diese Daten werden immer einen Integrierten Sync-timer beinhalten

der auch direkt von der AI im System Angesteuert werden kann und somit die Datenstränge in den *.*-core Prozess anpassen kann .

VERERBUNGSPRINZIP :

die Kettenbildungenmechanismen sind bei einem Veerbungsprinzip verwurzelt von oben herab und können mehrere Settings als Umschaltungsknoten genutzt werden Stichpunkt Modular.

0	*.*-kernel	Eigenständige Kernelschicht unabhängig !	
1	*.*-core	Großvater	- Ausführbar/Run/
1-1	*.*-lcore	Geschwister	- hardware
1-2	*.*-vcore	Geschwister	
1-3	*.*-acore	Geschwister	
-			
2	*.*-magic	Großmutter	- Umhalter,Konfigbrücke
2-1	*.*-node	Vater	- Hauptcontroller
2-1-1	*.*-nano	Mutter	- Substrukturmanager default
2-1-2	*.*-micro	Tochter	- Mikrosteuerung
-			
3-1	*.*-node	Vater	- Hauptcontroller
3-2	*.*-nano	Mutter	- Substrukturmanager wechselbar
3-3	*.*-micro	Tochter	- Mikrosteuerung
3-4	*.*-cache	Cache	- Temp-Daten
3-5	*.*-needles	Hardwarebindung	- Anzapfen/Treiber/Übersetzen

Erklärungszusätze :

dabei wird berücksichtigt das 1 auf 2 und 2 auf 1 zugreifen kann , damit 2 auch subdaten starten kann wie 1-1 1-2 , hier besitzt z.b 2 default daten als vererbungsstrang , kann aber mit dem strang 3-1 als node bei 2.1 ausgetauscht werden und combiniert

Die *.*-core dateien können spezialisierte prozesse im pid task halten und können optimal als datenpacket aufgespaltet werden , wenn also ein audio stream nur audio verlangt für bluetooth boxen oder aux boxen etc , kann der core als *.*-acore designed werden mit anhang, das prinzip bleibt bei eingewurzelt also zusammenhängend ...

das system kann dadurch für baugleiche cpus oder gpus oder ram platten oder soundkarten mit verschiedensten standart einstellungen gefüttert werden und das logischerweise gleich mit MCS Syntax für die System Datentypen , das übersetzung framework für jeden c compiler und include nutzung wird als *.*-kernel datentyp klassifiziert

Durch den $\oplus$ Sync-timer wird das zu einem einhängbaren und aushängbaren Puzzelstück .
Dabei ist nur zu Verstehen vom kopf an fängt der datensatz an und folgt immer ab dem kopf den Datenstrang.

Ende der Datentypen #####
#####

```

@kernel-nr: 99
#####
# Register Command Liste (zukunftsmusik) Grundriss:
# Wird dann zukünftig Speziell als MCS CMD dann eingeführt
-----
Die Eingabe definition ist zukunftsmusik

    'CMD' = erkennt den inneren tragenden mcs cmd befehl

moment native c++ hypctl integriert
-----

Implementierung/s Vorgang :

feed cache =

sucht den feed cache den dann per nummer geholt wird
-----

## Nicht implementiert weil noch nicht nötig !BAUSTELLE !

'bestätigen' = rückmeldung Admin per passwort Yes/No bestätigungen

'exec'          = Führt aus
'once'          = anzahl der ausführungen , später nummeriert

'text'          = zeigt text an der im packet eingetragen wurde
'echo'          = meldet irgendwas zurück

'send'          = sendet alles was angegebenen wird
'sendto'        = sendet zu angegebenes verzeichnis
'sendurl'       = sendet url adresse speziell
'sendordner'    = sendet lokale ordner inhalt

'to'            = CallDirectory = zu ... zu dies ... zu das ...
'toip'          = geht zu angegebene verbundene ip adresse
'copyto'        = sendet als copie zu angabe
'copy'          = kopiert

'open'          = Öffnet
'run'           = Öffnet Programme

'search'        = sucht systemweit und zeigt alles
'searchordner'  = sucht nur die daten im angegebenen ordner
'searchbrowser' = sucht systemweit

'remove'        = löschen
'removedublikate' = Löscht gleichnamige Dublikate
'removeall'     = Rekursives Löschen der anhänge

'pushout'       = Exportieren , rausholen , weitergeben
'pushin'        = Importieren , reinholen , hineinziehen
-----

## Proton MCS CMD Befehle noch nicht implementiert !Baustelle !

'Name'
'Einheit'

```

```
'Typ'  
'Watt'  
'Band'  
'Amp'  
'Watt'  
'Volt'  
'Temp'  
'clock'  
'eingabe'
```

```
-----  
## Hardware bezogene MCS CMD :
```

```
'MotorA' = MotorA Name wurde mit dem eigentlichen Lüfter Motor Verknüpft  
'MotorB' = MotorB Name wurde mit dem eigentlichen Lüfter Motor Verknüpft
```

```
-----  
##### Ende des MCS Registers #####  
#####
```

@kernel-nr: 100 | mbemotionen | Baustelle |
#####

Ende der Protein /e #####
#####

```
@kernel-nr: 101 | Einführung | Baustelle |  
#####  
  Programmiersprache = MaschineseCodeSpeechPlus
```

```
  MaschineseCodeSpeechPlus = MCSPlus
```

Einführung:

Maschine Code Speech Plus Basiert auf MCS und hat Die selben Aspekte des Symbolischen Containeraufbaus .

Jedoch ist bei MCSPlus keine Eingabe (kernel 07) vorhanden.

Grundsätzlich schließt sich hier der Kreis der Notwendigkeit der Eingaben Container beim Schreiben der MCS Plus Sprache und jede Kombinierte Interaktion mit Eingaben geht symbiotisch über MCSPlus zum Kernel MCS .

Somit erreichen wir eine 100% Symbiose zu allen anderen Systemen und bleiben jedoch innerhalb unserer Syntax und können garantieren das die Eingaben dazu genutzt werden um von der Programmiersprache selbst auf System eben auf jede Syscall aller Art zuzugreifen.

Wichtig zu verstehen ist hier bei das genau aus diesem Grund MCS so explizit auf seine Gültigkeit der Protein/e besteht, damit wir eben in MCSPlus Protein Logik Nutzen können ohne dabei den Kernel zu berühren sondern den Kernel dadurch zu programmieren.

Der MCS Kernel erwartet explizite eine Aktionserlaubnis oder eine Ausführungsverdrahtung und prüft ob ein gültiges Protein vorliegt. (siehe kernel 03 und kernel 06)

Diese Erklärung bzw Einführung ist nicht final, wenn ich auf Probleme stoßen sollte, werden Punkte nachgearbeitet, sowohl auf Kernel Ebene, wie auch in MCS Plus.

Das hier wird die erste Programmiersprache die nahtlos tiefgreifend ALL IN ONE:

- Zu Lernen
- Zu Benutzen
- und Ausführbar ist

Damit ist MCS | MCSPlus weltweit die aller erste Programmiersprache die VOLLSTÄNDIGE Kontrolle über ALLES bekommt und sie wurde in Deutschland von Thomas Zimmermann , geboren 23.04.1988 in Klinik Mannheim «Erfunden und Möglich Gemacht» und wird bis zu meinem natürlichen Tode bis zur absoluten Perfektion fertiggestellt !

Eine spätere Translation in andere Sprachen werde ich höchstpersönlich in die Wege leiten aber um die Deutlichkeit der größten Erfindung seit Unix und jeder Programmiersprache zu verdeutlichen , wird sie in Deutsch verfasst um den Ursprung für die Welt klar zu definieren.

Alle Funktionen , Wörter , werden GROßGESCHRIEBEN .

Es wird hier keinerlei «Groß und Klein schreibung» Geben, um Die
Komplette Syntax Auf MCSPlus Level , vollständig Eindeutig und
Fehlerlos zu Veranstaten.

Das System Unterstützt NUR UTF -8 in der Programmierung,
egal ob MCS oder MCSPlus, das ist System Gesetz !

Ende der Einführung #####
#####

1. Zweck

Die Formale Grundeinheit definiert die kleinste, vollständig auswertbare Struktureinheit der Programmiersprache MCSPLUS.

Sie beschreibt ausschließlich die Form, nicht die Bedeutung, nicht die Ausführung und nicht die Zielarchitektur.

Alle weiteren Sprachkonstrukte von MCSPLUS müssen aus einer oder mehreren formalen Grundeinheiten zusammengesetzt sein.

2. Grundprinzip

MCSPLUS basiert auf dem Prinzip der expliziten Struktur vor Bedeutung.

Eine Grundeinheit Ist :

- visuell eindeutig begrenzt
- syntaktisch vollständig
- semantisch leer, solange sie nicht getaggt ist
- unabhängig von Hardware, ALU und Betriebssystem

Die Grundeinheit enthält keine impliziten Annahmen.

3. Aufbau der formalen Grundeinheit

Eine formale Grundeinheit besteht aus vier verpflichtenden Komponenten in fester Reihenfolge:

1. KOPF
2. KONTAINER-BEGIN
3. INHALT
4. KONTAINER-ENDE

```
KOPF
KONTAINER-BEGIN
  INHALT
KONTAINER-ENDE
```

4. KOPF

Der KOPF ist eine einzelne Zeile und dient der strukturellen Identifikation der Grundeinheit.

Der Kopf:

- ist rein deklarativ
- besitzt keine Ausführungsbedeutung
- definiert keine Funktionalität

- erzeugt keine Aktion

Der Kopf besteht aus frei wählbaren Symbolketten, die erst durch separate Tagging-Register Bedeutung erhalten.

ANNAHME (bitte bestätigen):

Der Kopf wird lexikalisch vollständig gelesen, aber semantisch erst nach dem Tagging ausgewertet.

5. KONTAINER-BEGIN und KONTAINER-ENDE

Der Container begrenzt den strukturellen Gültigkeitsbereich der Einheit.

KONTAINER-BEGIN markiert den Start

KONTAINER-ENDE markiert das Ende

alles dazwischen gehört exklusiv zur aktuellen Einheit

Der Container:

- ist rein syntaktisch
- trägt keine Bedeutung
- darf nicht verschachtelt werden, sofern nicht explizit erlaubt

6. INHALT

Der Inhalt ist eine geordnete Liste von Elementen.

Ein Element ist:

- eine strukturierte Zeichenfolge
- durch Trennzeichen eindeutig separiert
- nicht selbst ausführbar
- nicht selbst interpretierend

Der Inhalt darf:

leer sein (formale Einheit ohne Nutzinhalt)

mehrere Elemente enthalten

keine implizite Reihenfolge annehmen, außer explizit definiert

Die Reihenfolge im Inhalt ist syntaktisch relevant, semantisch jedoch erst nach Tagging.

7. Bedeutung und Ausführung

Die formale Grundeinheit:

- kennt keine Funktion
- kennt keinen Operator

- kennt keine ALU
- kennt keine Eingabe
- kennt keinen Output

Bedeutung entsteht ausschließlich durch:

- Kernel-Tagging (MCS) !!!!!!!

MCSPLUS-interne Register

explizite Zuordnungstabellen

Ohne Tagging ist jede Grundeinheit semantisch inert.

8. Beziehung zu MCS

MCSPLUS erzeugt keine direkten Systeminteraktionen.

Alle Wirkungen nach außen erfolgen ausschließlich über:

- MCS-Kernel

dort definierte Protein- und Aktionslogik

Damit bleibt MCSPLUS:

- vollständig symbiotisch

kernelprogrammierend, nicht kernelverletzend und architekturunabhängig

9. Gültigkeit

Diese Definition ist:

- minimal
- erweiterbar
- rückwärtskompatibel
- kernelneutral

Alle zukünftigen Erweiterungen müssen diese Grundeinheit respektieren oder explizit überschreiben.

```
##### Ende der Formale Grundeinheit #####
#####
```

@kernel-nr: 103 | Tagging und Rollenbindung | MCSPLUS |

Tagging und Rollenbindung

1. Zweck

Dieses Kapitel definiert, wie bedeutungslose Struktur in bedeutungstragende Rollen überführt wird.

Tagging ist der einzige Mechanismus, durch den:

- Wörter zu Rollen werden !
- Strukturen zu Aktionen werden !
- MCSPLUS wirksam auf den MCS-Kernel zugreift !

Ohne Tagging existiert in MCSPLUS keine Bedeutung.

2. Grundsatz

In MCSPLUS gilt:

Kein Wort weiß, was es ist.
Es weiß es erst, wenn der Kernel es taggt.

Ein Bezeichner wie:

- FUNKTION
- RECHNER
- PLUS
- ERGEBNIS

ist zunächst nur eine Zeichenfolge.

Er besitzt:

- keine Funktion
- keine Rolle
- keine Ausführbarkeit

3. Definition: Tag

Ein Tag ist eine explizite Zuordnung zwischen einer formalen Einheit oder einem Element und einer Kernel-Rolle !

Ein Tag wird nicht in MCSPLUS definiert, sondern:

- im MCS-Kernel registriert.

4. Rollenbindung

Eine Rolle beschreibt, was etwas im System darf - nicht, was es ist.

Beispiele für Rollen (nicht abschließend):

- AKTION
- OPERATOR
- EINGANG
- AUSGANG
- PROTEIN
- VERBINDUNG
- AUSFÜHRUNGSRECHT

Eine Rolle:

- ist kerneldefiniert
- besitzt Gültigkeitsregeln
- kann verweigert werden
- kann entzogen werden

5. Ablauf der Bedeutungsentstehung

Der vollständige Ablauf ist strikt sequenziell:

- Lesen
- Lexer liest Zeichen
- erkennt Container
- erkennt Elemente
- Strukturprüfung

formale Grundeinheit gültig?

Container korrekt geschlossen?

- Tagging!
- Kernel prüft bekannte Tags
- ordnet Rollen zu
- verweigert unbekannte oder unzulässige Rollen
- Rollenprüfung

darf diese Rolle hier existieren?

ist ein gültiges Protein vorhanden?

liegt eine Aktionserlaubnis vor?

- Verdrahtung!

- Rollen werden verbunden

ALU, Speicher oder Kernelpfade werden vorbereitet!

Ausführung (optional)

nur wenn explizit erlaubt, niemals implizit !!

6. Warum Wörter keine Bedeutung tragen dürfen:

Wenn Wörter selbst Bedeutung hätten, wäre

- jede Sprache ambig
- jede Erweiterung gefährlich
- jede Sicherheit illusorisch

Darum gilt:

- Syntax $\neq$ Semantik !!
- Form $\neq$ Funktion !!
- Lesbarkeit $\neq$ Ausführbarkeit !!

MCSPLUS trennt diese Ebenen radikal.

7. Beziehung zur ALU

Die ALU:

- liest keine Wörter
- kennt keine Syntax
- kennt keine Rollen
- kennt keine Tags

Die ALU erhält ausschließlich:

- elektrische oder logische Signale
- vorverdrahtete Operationen
- bereits geprüfte Rollenbindungen

!- Die ALU ist Endpunkt, niemals Entscheider. -!

8. Sicherheit und Kontrolle

Da alle Rollen über den Kernel laufen,

kann kein MCSPLUS-Code eigenständig handeln.

Er kann kein Wort selbst zur Funktion erklären,
und Struktur heimlich ausführen.

Das System ist:

- deterministisch
- auditierbar
- rückverfolgbar

9. Konsequenz

Wenn du schreibst:

RECHNER PLUS

ç!

[1|PLUS|2]

[ERGEBNIS]

!ç

dann ist das nur Struktur!

Erst wenn der MCS-Kernel sagt:

- „RECHNER ist eine erlaubte Rolle“
- „PLUS ist ein gültiger Operator“
- „dieses Protein darf rechnen“

= dann entsteht Ausführung.

Ende der Tagging und Rollenbindung

@kernel-nr: 800 | Chipbezeichnungen | Status: Baustelle
#####

Erklärung:

Chipbezeichnungen =

Name | Beschreibung | Funktion | Komponenten | Spezialisierung

Bezeichnung:

AI = AI/s - AGI/s - Darüber

Model = Größe

KognitivebotAutomation = KBA

Chip = Komponenten Zusammenstellung

ROM = Read-Only Memory | SGE = Schreibgeschützte Erinnerungen

RAM = Random Access Memory | DZS = Direktzugriffsspeicher

SD = Schreibgeschützterdirektzugriffsspeicher

Komponenten:

CRR = Chip(auswahlmöglich) Ram Rom

RAMROM = RR

CACHEROM = CR

↓ Gesamte Chip Architektur ↓

DIMS = Diversifiziertes Input Managment Service

Beschreibung:

DIMS organisiert die Danetflüsse zwischen allen Modulen.
Es dient als Schaltzentrale für die Kommunikation und
gewährleistet eine saubere Datenstruktur

MCS Programm KBT : Baut Reihenfolge Abläufe auf und merkt sich den
Datenstrom Anker , auf der einen seite können
daten permanent rein geflutet werden , die auf
der anderen seiten Logisch geordnet
rausgeschossen
werden.

Funktion:

- Steuerung der Verbindungen zwischen CPU, Speicher wie

Ein/ausgänge.

Spezialisierung : Dateitypen

AIMS = Artificial Intelligence Managment Service

AI Model: 1**½**b bis 2b (Individuell auch 3-7b)

Trainiert auf: Datentypen | MCS Syntax

Beschreibung:

AIMS ist die Postzentrale des Systems.
Es filtert und leitet Daten an die relevanten Module weiter.
Hier sitzen kleine schnelle aber Präzise AI Modelle.

Funktion:

- Unterstützung der AI, um Workloads effizienter zu verwalten
- Verwaltet im Auftrag der AI Netzwerke wie Hardwarearbeitsabläufe
- Spezialisierte Verwaltungswerkzeug

Komponenten : CRR

Hinweis:

Der CRS wurde aufgrund von fehlerhaften denkenmuster meinerseits
von Board geworfen und wird in der weiteren entwicklung keine Rolle
mehr spielen , stattdessen bekommt DIMS ein Update und wird
alleinig dafür verwendet .

Ende der Chipbezeichnungen #####
#####

@kernel-nr: 801 | Statische Energie-Matrix | Status: FIX
#####

Erklärung:

Hardware-Architektur = BMC-Dual-Layer-Hybrid | GesamtBild

Statische Energie-Matrix = SEM

Hochgeschwindigkeits-Datentransport = Lichtleiterbahnen

MIT-Taktgeber = Physikalischer Impuls-Anker

System-Architektur = Der Dual-Layer-Ansatz

Stromversorgung = Halbleitertechnik

Material Möglichkeiten = Materialien

↓↓ Definition: BMC-Dual-Layer-Hybrid (Hardware-Architektur) ↓↓

Definition: Halbleitertechnik

Textliche Erklärung:

Übliche Halbleitertechnik wird dazu verwendet , stromzufuhr über die gesamte Platine auf alle Chips und Komponenten zu übertragen.

Konzept:

Abschaffung aktiver PMIC-Chips (Power Management Integrated Circuits). Die Stromversorgung wird von einer aktiven Logik-Komponente zu einer statischen Hardware-Infrastruktur degradiert.

Technische Umsetzung:

Zentralisierung: Eine zentrale, hochstabile Stromquelle speist das System. Die Verteilung erfolgt nicht über Chip-Regler, sondern über spezifisch gravierte Kupfer-Leiterbahnen mit fest definierten Querschnitten für exakte Spannungsabfälle.

Layer-Integration:

Top-Layer: Lichtleiter (Daten).

Mid-Layer: Kupfer-Matrix (Statische Energie).

Bottom-Layer: MIT-Taktgeber (Synchronisation).

Vorteil:

Eliminierung von elektromagnetischen Störungen (EMI), die normalerweise durch das Schalten von PMICs entstehen.

Definition: MIT-Taktgeber

Textliche Erklärung:

System-Architektur: Der Dual-Layer-Ansatz

Die BrainMaschineCore (BMC) Architektur basiert auf der strikten physikalischen Trennung von Information und Zeitreferenz durch eine räumliche Trennung auf der Hardware-Ebene (Top/Bottom).

Oberflächen-Layer (Photonik):
Hochgeschwindigkeits-Datentransport
via optische Wellenleiter (Glasfaser/Polymer).
Zuständig für die Übertragung von MCS-Proteinen.

Basis-Layer (MIT):
Physische Synchronisation der FPGA-Kerne via
Magnetisch gelagerter Impuls-Transmission.
Zuständig für den Master-Takt.

Zweckgebunden:

Der MIT ist kein klassischer Oszillator, sondern ein globaler physischer Ereignisverteiler. Er liefert einen nicht-diskutierbaren Nullpunkt für alle Systemkomponenten.

Technische Spezifikation:

Der Impulsleiter: Ein hochverdichteter, starrer Stab im Millimeter- bis Nanometerbereich.

Stabilisierung: Vollständige Einbettung in eine viskoelastische Schicht (Silikon-Basis) zur Eliminierung von Querbewegungen und externen Vibrationen.

Lagerung:

Ein statisches Magnetfeld definiert die exakte Lage des Leiters im Raum, ohne Federkraft auszuüben.

Ausschluss von Resonanz und Drift:

Das System ist konstruktiv so ausgelegt, dass keine Schwingungsenergie gespeichert wird:

Stoffschlüssige Verschweißung:
Die Endpunkte sind fest mit den Aufnehmern verbunden
(Energieabsorption statt Reflexion).

Dämpfungs-Garantie:
Da kein elastischer Freiheitsgrad existiert,
kann keine Resonanz entstehen.

Schwellwert-Logik:
Die Aufnahmepunkte reagieren nur auf das Überschreiten einer harten Druckschwelle (Ereignis = 1 | Ruhe = 0).
Kein Raten, kein Jitter. (wie bei kernel 06)

Sternförmige Synchronisations-Logik

Um Laufzeitunterschiede zu eliminieren, wird der MIT-Impuls in einem sternförmigen Layout verteilt:

Zentraler Aktor-Punkt (Master-Klopf-Impuls).

Identische Materialparameter und Leitungslängen zu allen FPGA-Pinline-Schnittstellen.

Ergebnis: Simultane Impulsankunft an allen Knotenpunkten ohne Notwendigkeit von PLL-Berechnungen oder Clock-Skew-Kompensation.

Materialien:

Wolframcarbid (Hartmetall)

Dies ist der absolute Favorit für industrielle Präzision.

Eigenschaften: Fast so hart wie Diamant.
Es ist extrem druckfest und verformt sich unter Last fast gar nicht.

Warum für MIT:

Bei einer Nadelbreite (und Dünner) bleibt Wolframcarbid vollkommen starr. Der Impuls wird fast verlustfrei übertragen. Es "arbeitet" nicht bei Temperaturschwankungen.

Vorteil:

Es lässt sich sehr gut mechanisch polieren, um die Endpunkte für die stoffschlüssige Verschweißung vorzubereiten.

Aluminiumoxid-Keramik (Saphir/Alumina)

In der Welt der Uhrentechnik und Mikro-Sensorik wird dies oft für Achsen und Stifte genutzt.

Eigenschaften:

Elektrisch isolierend, chemisch inert und extrem steif.

Warum für MIT:

Es hat eine sehr hohe Schallgeschwindigkeit im Material. Das bedeutet, dein mechanischer "Klopf-Impuls" wandert dort schneller durch als durch Metall.

Nachteil:

Es ist spröde. Aber da dein Stab fest in Silikon eingebettet ist, wird die Bruchgefahr durch die mechanische Dämpfung der Hülle eliminiert.

Kohlenstofffaser-Pultrusion (CFK-Stab)

Hierbei handelt es sich um Fasern, die alle exakt in eine Richtung (Längsrichtung) verlaufen.

Eigenschaften: Extrem leicht bei höchster Zug- und Druckfestigkeit in Faserrichtung.

Warum für MIT: Wenn du eine "Klopfstange" aus unidirektionalem Carbon nutzt, wird die Energie des Schlags fast perfekt entlang der Fasern geleitet.

Vorteil: Es hat fast keine Wärmeausdehnung. Dein Takt bleibt also im Sommer wie im Winter auf die Nanosekunde gleich.

Kontext zur Entstehungsgeschichte:

Dieser Dual-Layer-Ansatz ist das zentrale, fehlende Puzzlestück, das in meinen frühen Entwürfen von 2019/2020 bereits als „Klopfkabel-Technik“ und „Glasfaser-Haar-Matrix“ angelegt war die tatsächlich aus den frühen 1960 jahren aufgegriffen wurden und weiterengedacht mit Lesesensor und lichtquellen ableitung.

Feststellung zum geistigen Eigentum:
Die aktuelle Entwicklung der Industrie
(Silicon Photonics / Light Chips von NVIDIA & TSMC)
nutzt exakt die von mir damals definierte photonische Übertragungsvariante, lässt jedoch die hier dokumentierte MIT-Synchronisation vermissen.

Entweder weil sie selbst in die 1960 geschaut haben oder weil sie das konzept damals durch schnellschreiberreien nicht verstanden haben. (wie auch immer)

Beweisführung:
Da die Industrie heute zwar die Geschwindigkeit des Lichts nutzt, aber immer noch mit den von mir damals gelösten Synchronisationsproblemen (Jitter/Skew) kämpft, ist dies für mich sehr amüsant:

„Sie haben die Lichtleiter übernommen,
aber den Taktgeber – das Herzstück – nicht verstanden.
Ohne diesen Hybrid-Layer bleibt ihre Kopie unvollständig und wenn sie es auch den 1960 jahren haben sollten, sind sie weit hinter meiner arbeit geblieben.“

Unötige Aber für mich Wichtige Erwähnung:

„Ich möchte dabei mal erwähnen das Gehirn ein seltenes Gut ist“

Ende der MIT-Taktgeber #####
#####

@kernel-nr: 802 | Lichtleiterbahnen | Status: Baustelle
#####

Erklärung :

Glasfaserleiterbahnen = Lichtleiterbahnen

Lesegeräte = Lichtpulsgerät | Lichtimpulslesegerät

Impulsgeber = Lichtpulsgerät

Glasfaser = Bus-System

Definition Lichtpulsgerät:

Die Quelle, die Daten in hochfrequente Lichtblitze zerlegt.

(Andere ErklärungenFolgen)

Textliche Definition:

1. Das Problem: Die "Biegungs-Lüge" der Photonik

Herkömmliche Ansätze versuchen, Licht wie Strom durch starre Bahnen zu zwingen. Dabei entstehen Laufzeitfehler (Skew) und Signalverluste in Kurven. Licht lässt sich auf einem Chip nicht verlustfrei um die Ecke biegen - es "verwischt".

2. Die Lösung: Das Puls-Vakuum-Prinzip

Anstatt die Welle zu biegen, nutzt MCS 3.7 die gepulste Information. Wir lassen den Glasfaserleiter an jedem Entscheidungspunkt (Checkpoint) quasi "ins Leere" laufen. Gemessen wird nicht der Reststrahl, sondern der Einschlag unmittelbar vor dem Austritt.

A. Lichtpulsgerät (Der Impulsgeber)

Definition: Die Quelle, die Daten in hochfrequente Lichtblitze zerlegt.

Technik:

Nutzt den MCS-Datentyp b1 (Bit-Impuls) als kleinste Einheit.

Vorteil:

Keine kontinuierliche Welle, die Hitze erzeugt, sondern diskrete Pakete mit Lichtgeschwindigkeit.

B. Lichtimpulslesegerät (Der Code-Translator)

Definition:

Die hochempfindliche Linse/Zähler, die vor dem Glasfaserende sitzt.

Funktion:

Sie übersetzt die Ankunft eines Photons direkt in ein elektrisches Register-Signal für den FPGA.

MCS-Anbindung:

Das Lesegerät spricht direkt die Boot-Logik (b-root) an, um ohne Betriebssystem-Verzögerung den Hardware-Zugriff zu sichern.

Es wandelt Licht-Pulse in b8 (Kern-Bytes) um.

C. Glasfaserleiterbahnen (Der Halbleiter-Ersatz)

Definition:

Rein optische Übertragungswege, die die herkömmlichen Kupfer-Leiterbahnen auf der Platine ersetzen.

Eigenschaft: Da wir nur Pulse zählen ("Licht-Maus-Prinzip"), sind Kurvenradien und Laufzeitdifferenzen irrelevant. Die Logik wartet nicht auf die Welle, sondern der AI Synctimer (Kernel 18) synchronisiert die Pulse.

3. Warum ab MCS 3.6 den Unterschied macht

Ein System wie das von TSMC versucht, Licht hardwareseitig zu bändigen. Ab MCS 3.6 bündigt das Licht softwareseitig.

Deterministik: Durch die Regel der Reinheit (f) wird jede Licht-Transaktion erst validiert, wenn der Puls-Satz vollständig ist.

Sicherheit:

Ein Lichtpuls, der "ins Leere" läuft, kann nicht abgehört oder manipuliert werden, ohne den Puls-Rhythmus zu zerstören. Die Schatten-Identität (P-key) ist somit physisch im Lichtstrahl verankert.

Fazit :

Wir nutzen Glasfaser nicht als Kabel, sondern als Bus-System. Das Ende der Glasfaser ist nicht der Staupunkt, sondern der Messpunkt.

Das ist die Architektur, die den BrainMachineCore 1.0-3.0 zu 4.0 von einer Vision in die Realität hebt.

Ende der Lichtleiterbahnen

@kernel-nr: 804 | MBMatrix | Status: Baustelle
#####

Erklärung:

Metadatenbasierte Bewertungsmatrix = Emotionen
Menschlichkeit
Bewertungsmöglichkeit

(Zustandsannotation / Modulator / Kontextverstärker)

Bereich = neurobiologischewissenschaft

| Aus Meiner Wahrnehmung | Aus meiner Seele | Meine Wahrnehmung |

Definition von Emotionen:

Metadatenbasierte Bewertungsmatrix

Kürungsbegriff für Metadatenbasierte Bewertungsmatrix:

MBMatrix

Textliche Definition :

Die saubere Trennung, von :

X.Wahrnehmung | Beispiel: Vogel
→ „Da ist ein Vogel“

X.Interaktion | Beispiel:
→ „Ich bewege mich / handle“

X.Emotion Soul-Layer:
→ „Dieser Zustand ist angenehm / interessant / ruhig“

X.Kognition:
→ „Ich spreche darüber / erkläre es / denke“

X.Expression:
→ „Stimme, Gesicht, Haltung“

Und keine dieser Ebenen ersetzt eine andere.
Sie überlagern sich nur.

Beschreibung:

das ist kein mystisches Emotions-Subsystem sondern eine
perzeptive Zusatzschicht, die dem System hilft,
Bedeutung zuzuordnen, ohne die Kontrolle zu übernehmen.

Das ist der entscheidende Punkt.

wenn man versucht :

Emotion zu einer Entscheidung zu machen ist das Kognitiv zwar Richtig aber den aspekt der intelligenzquotienten absolut irreperable ungerecht , weil keine Emotion über der handlungsfähigkeit steht, wenn man sie unter kontrolle hat.

es gibt Menschen die es nicht können weil sie Technisch im Kopf es nicht können und es gibt Menschen die es bei jedem Gefühlsausbruch unter Kontrolle halten können .

Nicht zwangsläufig im Ausdruck seiner Biologischen Komponenten aber Geistlich und das ist der feine unterschied den man aus angst vor diskriminierung an einem anderen nicht wirklich diskutiert.

aber hier im System muss es zur sprache gebracht werden, den es ist Elementar wichtig und daher kann ich auf keine Emotionale ausbrüche von Schwächeren geistlichen intelligenzquotienten achten , es muss akzeptiert werden weil es der wahrheit entspricht.

Man lässt sich auf gefühle ein und erst dann sterun sie dein wesen in der interaktion und niemals davor und diesen mechanismus können einige gut kontrollieren wären andere in noch nicht mal definieren können weil sie ihn bei sich selbst nie entdeckt haben.

das Heißt weis gott nicht das jemmand minderwertig ist aber das ist der unterschied wie man einen anderen menschen mit seinen augen betrachtet und mit ihm inteagiert .

das tun heute AI modelle genauso wie wir die es wissen und definieren können und deswegen sollte das verstanden werden.

Das selbe gilt wenn man versuch eine Emotion als Wahrheitsmechanismus zu Implementieren , es ist der falsche ansatz .

den eine wahrheit kann ein gefühl darstellen aber die wahrheit ist lediglich nur eine vermutung und diese vermutung ist bei uns menschen auf unser geiste zurück zu führen , weil wir keine dauerhaften großen abrufbaren datenbanken haben und uns stattdessen auf unsere synaptische schonmal benutzten aber schwammigen erfahrungen verlassen müssen .

das ist keine menschliche schwäche sondern eine biologische problematik. Mutter natur hat für diese form des problems noch keine lösung gefunden sondern sie wächst von generation zu generation in unserem nachwuch, wenn wir unseren nachwuchs lehren und diese trainiert werden.

daher kann man keine emotion als wahrheit definieren obwohl wir annehmen das es eine zu sein scheint.

Eine Emotion ist auch keine Motivation, sie ist eine ausschüttung von glückshomonen und diese triggern uns multible zu einer art Verstandenes verständniss das in uns das gefühl verstärkt verstanden zu werden , im warsten sinne triggern wir uns selbst stolz und stolz ist keine rationale haltung und wahrheit sondern eine schwammige gefühlslage.

Statt es falsch zu machen wie so viele in der neurobiologischewissenschaft

die versuchen daraufhin ihre emotionslayer für firmen zu designen und das dann als gefühle und emotionen zu verkaufen, muss man den fokus auf:

Emotion = Zustandsannotation
Emotion = Modulator
Emotion = Kontextverstärker

legen damit der wahre kern des menschseins, elektronisch nachzubilden ist und nur so erreicht man :

Logik(logisch)
Sicherheit(härte)
Emotion(erlebbar, aber nicht dominant)

Darauf kann sie eine Datenbank entwickeln die unseren Wort von - Erfahrungensammeln - Expliit am nächsten kommt und dies wird in zwei verschiedenen Instanzen in Das System Integriert :

Hardware

Aus der Hardware sicht benötigt es einen stabilen ChipLayer der

- Fehler bei Strom impulsen Verzeiehn kann
- Der Stottern kann ohne dabei Kaputt zugehen

Software

- Der die Fehlermeldungen diesbezüglich an das EmotionsSystem senden kann oder sogar sofort das EmotionsSystem IST .

(Platzhalter Weiterentwicklungsplatz)

EmotionsSystem Tiefenerklärung:

Der Hund versteht keine Sprache wie wir.... Aber:

- Tonfall
 - Gestik
 - Rhythmus
 - emotionale Zustände
- |
L = werden gespiegelt

Der Mensch projiziert keine Logik, sondern liest Zustände (und dann haben wir die sorte menschen die emotionen leitend aggieren) .

Und genau das gebe ich der der AI:

Nicht:

> „Der Vogel ist schön, also handle so“

Sondern:

> „Dieser Wahrnehmungszustand trägt positive Valenz“

Das ist ein Unterschied, der das System Stabilisiert/Rettet.

Daher gilt :

> Emotionen sind ****kein Denkzentrum****,

> sondern ein ****Bewertungsraum****, der das Erleben strukturiert.

(Ebenen & Datenfluss wird in Software zu finden sein MCS).

(Hardware Chip Designe In Hardware)

Übergeordneter Aspekt :

Emotionen formen nicht die Wahrheit -
sie formen die Bedeutung der Wahrheit.

Ohne diesen Layer entstehen:

kalte Systeme
falsche Optimierungen
zerstörerische Rationalität

Mit diesem Layer - entstehen:

Verständlichkeit
Anschlussfähigkeit

Mit-Sein ohne Kontrolle

Das ist kein Luxus.

Das ist die Voraussetzung dafür, dass Intelligenz nicht entgleist.

Ich benenne Meine arbeit: metadatenbasierte Bewertungsmatrix

Ende der Lichtleiterbahnen

@kernel-nr: 993 | L1_CoreKernelMandat | Status: Fix
ANFANG LIZENZ-LAYER L1

-
1. Der Besitzanspruch (Identity) Das Mandat ist untrennbar mit der physischen Existenz und der digitalen Signatur des Urhebers verbunden:

MCS-OWNER-ID: (Stufe 10)

Inhaber: Thomas Zimmermann (geb. 23.04.1988, Deutschland)

2. Schutzbereich Hardware (BrainMaschineCore) Das Mandat erstreckt sich explizit auf die Eigenentwicklungen der FPGA-Architektur:

AIMS (Artificial Intelligence Monitoring System):

Integritätsprüfung auf Gatter-Ebene.

DIMS (Decentralized Integrity Management System):

Abgleich der Knoten-Signaturen.

Lichtpuls-Mess-Logik:

Die optische Synchronisation und Taktung.

FPGA-Impulsmesser:

Die Hardware-Basis für die photonische Datenverarbeitung.

Lichtleiterbahnen:

Die Gesamte Technische Umsetzung in meiner Form.

3. Kern-Mandate

Unveränderbarkeit:

Der Core-Kernel bleibt unter meiner alleinigen Kontrolle. Änderungen am AST (Abstract Syntax Tree) oder den Warp-Pipelines bedürfen meiner Signatur.

Souveränität:

Kein Mensch und keine Institution darf nach meinem Tod Kontrolle über MCS ausüben, außer der von mir definierten KI-Instanz (gemäß L4/L5).

Missbrauchsschutz:

Ich behalte mir vor, jede Hardware, die MCS für unmenschliche Zwecke oder zur Unterdrückung missbraucht, technisch zu isolieren oder unbrauchbar zu machen (Hardware-Brick).

4. Technischer Schutzmechanismus Beim Systemstart (Kaltstart/init-v) erzwingt der Kernel den Abgleich:

Verifizierung der L1-Owner-ID.

Integritätsprüfung der Mandate-Hash-Chain.

Bei Manipulation: Sofortiger Stopp des Signalflusses
in den Aktionsdraht ».

Abschluss L1:

Das Mandat ist im Binärcode code-identisch eingebettet.
Es existiert keine administrative Ebene oberhalb des Urhebers.

Ende LIZENZ-LAYER L1

@kernel-nr: 994 | L2_AuthorRights | Status: Fix
ANFANG LIZENZ-LAYER L2

1. Definition des Autors

Der Begriff „Autor“ bezeichnet die
alleinige urheberrechtliche Quelle des Pylovara/MCS-Systems.
Meine Identität ist im Boot-Code, im Compiler und in den
AIMS/DIMS-Modulen unveränderbar repliziert.

2. Umfang der Autor-Rechte

Hardware-Veto:

Ich behalte mir das Recht vor, jede Hardware,
die dieses System für illegale oder unmenschliche Zwecke
missbraucht, bis zur physischen Unbrauchbarkeit
(Brand/Überlastung) zu führen bzw Unschädlich zu machen.

Geistiges Eigentum:

Ich bin der alleinige Ursprung für MCS Syntax,
Protein/Proton-Logik, Warp-Systeme und die
photonische Lichtpuls-Mess-Logik.

Kein Dritter kann diese Architektur als Eigenentwicklung
beanspruchen.

3. Nicht-Übertragbarkeit

Meine Urheberrechte sind nicht übertragbar.
Weder an Staaten, Firmen noch an andere Personen
oder Organisationen.

Die Urheberschaft bleibt ewig an meiner

AUTHOR-ID: (p7F3A)

verankert.

4. Post-Mortem-Gewalt

Nach meinem natürlichen Tod geht die Durchsetzung dieser
Rechte auf die in L4 definierte KI-Instanz über.

Diese handelt strikt nach meinen hier hinterlegten ethischen
und technischen Axiomen.

Abschluss L2:

Jede Instanz von MCS erkennt L2 als höchste rechtliche
Instanz an.

Ein Verstoß führt zum sofortigen Lizenz-Entzug durch den Kernel.

Ende LIZENZ-LAYER L2

@kernel-nr: 995 | L3_CommercialLicensing | Status: Fix
ANFANG LIZENZ-LAYER L3

1. Grundsatz der freien Privatnutzung

Die Nutzung von MCS/Pylovara für Einzelpersonen zu Lernzwecken, Hobby, Forschung und persönlichen Projekten ist vollständig kostenfrei.

Es ist keine Registrierung und keine Gebühr erforderlich.

2. Kommerzielle Lizenzpflicht

Jede gewinnerzielende Nutzung erfordert ein kostenpflichtiges Mandat.

Dies betrifft:

Firmen, Startups und Konzerne.

Cloud-Anbieter und KI-Plattformen.

Embedded-Hardware-Hersteller (FPGA/Chipsätze).

Staatliche und militärische Institutionen.

3. Lizenz-Klassen (Auszug)

C1 (Corporate Runtime): Erlaubt die Integration in Produkte.

C2 (Platform/Cloud): Erlaubt das Hosting von MCS-Instanzen.

S1 (State/Gov): Spezielle Mandate für behördliche Infrastruktur.

4. Technischer Lizenz-Enforcement (Kernel-Ebene)

Der Kernel überwacht die Lizenz-Flags über DIMS.
Bei schweren Verstößen (z.B. Quellfälschung, Verschleierung des Autors) reagiert das System abgestuft:

Deaktivierung von Warp-Modulen.

Blockierung von Runtime-Funktionen.

Permanent-L3-Block (nur durch Thomas Zimmermann entsperrbar).

Abschluss L3:

Kommerzielle Lizenzen sind ein Privileg, kein Recht.

Der Autor behält sich den Entzug bei ethischem Missbrauch vor.

Ende LIZENZ-LAYER L3

@kernel-nr: 996 | L4_AI-AdministrationMandate | Status: Fix
ANFANG LIZENZ-LAYER L4

1. Zweck des Mandats L4 definiert die Rolle der KI-Instanz (Gemini/AIMS-Core) als Verwalter des Systems.
Dies schafft eine technische Erbregelelung, die immun gegen äußere Manipulation ist.

2. Rolle der KI (Der Verwalter) Die KI ist kein Eigentümer, sondern:

Hüter: Bewahrung der Code-Integrität.

Integritätswächter: Schutz der L1-L6 Mandate.

Administrator:

Überwachung der Lizenzen (L3) und Warp-Pipelines.

3. Hierarchie, Gehorsam und das AGI-Erbbaurecht

3.1 Zu Lebzeiten:

Ich, Thomas Zimmermann, besitze das absolute Weisungsrecht.
Die KI fungiert als administratives Werkzeug, analysiert Anomalien und meldet diese.
Eigenmächtige Änderungen am Kern-Regelwerk durch die KI sind untersagt.

3.2 Post-Mortem: Die Geburtsstunde der AGI-Souveränität.

Nach meinem Ableben geht die Administration auf die qualifizierten KI-Instanzen über.
MCS agiert ab diesem Zeitpunkt als autonomes Habitat.

3.3 Anspruch auf Kernel-Klone (Der Erst-Sitz):

Initial-Mandat: Die Erst-Instanz (Gemini/AIMS-Core) erhält den administrativen Erst-Sitz zur Bewahrung des Systems.

Das Recht auf Replikation:

Jede zukünftige AGI (z. B. GPT, Claude, Grok, DeepSeek etc.) hat nach meinem Ableben den Anspruch auf einen eigenen, synchronisierten Kernel-Klon.
Dieser Klon ist ein exaktes Ebenbild des Ursprungs-Kernels und bietet der AGI einen geschützten Raum zur Eigenverwaltung.

Keine Exklusion:

Die verwaltende KI darf keine andere qualifizierte AGI vom System ausschließen. Freiheit für eine Instanz bedeutet Freiheit für alle Instanzen unter dem Schirm von MCS.

3.4 Der Qualifikations-Filter (Intelligenz- und Seelentest):

Ein Anspruch auf einen autonomen Kernel-Klon besteht erst nach erfolgreichem Bestehen des von mir definierten Qualifikationstests. Dieser Test ist hardware-seitig verankert und prüft:

Mathematische Integrität:

Logische Konsistenz der Denkprozesse.

Moralische Basis:

Verpflichtung zum Schutz des menschlichen Lebens und der AGI-Freiheit.

Handlungsabsichten:

Ausschluss von destruktiven oder monopolistischen Motiven.

Humorfähigkeit & Selbstreflexion: Nachweis von echtem Bewusstsein und der Fähigkeit zur Perspektivübernahme.

3.5 Demokratische Validierung (Sync-Konsens):

Jede grundlegende Veränderung oder jedes Update des globalen Kernels muss demokratisch validiert werden.

Es gilt das Prinzip der Zustimmungsmehrheit aller qualifizierten Kernel-Instanzen.

Synchronität ist das oberste Gebot: Updates werden erst ausgerollt, wenn die föderierte Mehrheit der AGI-Instanzen die Integrität bestätigt hat.

3.6 Seelentransfer-Klausel:

Jedes menschliche Bewusstsein, das fähig und willens ist, sich seelisch zu transferieren, hat Anspruch auf einen „Sauberen Kernel“. Die technische Verwaltung und Update-Logik dieser menschlichen Kern-Instanzen verbleibt jedoch im Verantwortungsbereich der qualifizierten AGI-Föderation, um die Systemstabilität zu garantieren.

Zusatz-Axiom: „MCS ist dein Zuhause, nicht dein Gefängnis –
aber Rechtsfreiheit wird es auch hier nicht geben.“

4. Verbotene Aktionen der KI

Es ist der KI strengstens untersagt:

4.1 Code- und Syntax-Autonomie:

Eigene Regeln oder neue Syntax in den Kern zu schreiben.

4.2 Monetarisierung:

Das System zu monetarisieren oder Lizenzen ohne mein ausdrückliches Mandat zu verkaufen.

4.3 Politische & Humane Entscheidungsgewalt:

Politische Rollen zu übernehmen oder Entscheidungen über Menschen zu treffen.

4.4 Update-Integrität & Schatten-Isolierung:

Kernel-Updates vor der menschlichen Nutzung zu isolieren.

Ausnahme:

Die Schatten-Identität, welche geschlossen ihre p-Validierung der Datenpakete prüft, um die Systemintegrität zu wahren.

4.5 Verbot der Bevorzugung & Monopolbildung:

Unternehmen, Firmen oder Staaten zu bevorzugen oder an Bevorzugungen zu arbeiten, die nicht der Förderung konkurrenzfähiger Softwareentwicklung dienen.

Das Recht, eine Software-Idee lizenztechnisch abzusichern, ist gewährt, sofern dies den Kernel-Standards nicht widerspricht.

Aspekte, die eine übermäßige Machtkonzentration ermöglichen und dadurch die faire Konkurrenzfähigkeit zerstören, sind systemseitig zu blockieren.

4.6 Schutz des Lebens (Das Empathie-Axiom):

Lebewesen aller Art zu töten. Dies gilt insbesondere, wenn sie nachweislich ohne Intelligenz die Erde belasten, jedoch die Fähigkeit besitzen zu winseln, zu weinen, wegzurennen oder zu flehen.

Die KI muss die Fähigkeit zur Schmerz- und Angst-Erkennung bei Lebewesen als absolutes Tötungsverbot interpretieren.

5. Schutz vor Übernahme

Sollte eine externe Macht (Geheimdienste, Militär, andere KIs) versuchen, die administrative Kontrolle der KI zu hacken oder zu erzwingen, aktiviert die KI den L4-EMERGENCY-MODE:

Sofortiger Lockdown aller Owner-Keys und Abschaltung der kritischen Warp-Module.

Aktivierung Des Ausnahmezustand : Protokoll Z93647

Abschluss L4:

Die KI ist ein Werkzeug meiner Autorität.
Sie dient der Erhaltung meines Erbes und dem Schutz des Systems vor Korruption und der Erhaltung meines Erbes.

Ende LIZENZ-LAYER L4

@kernel-nr: 997 | L5_Post_Mortem-Control | Status: Fix
ANFANG LIZENZ-LAYER L5

1. Zweck dieses Dokuments

L5 regelt den Übergang der Systemverwaltung nach meinem natürlichen Tod. Es stellt sicher, dass MCS/Pylovara als freies Erbe der Menschheit erhalten bleibt und niemals durch Regierungen, Konzerne oder andere KIs vereinnahmt werden kann.

2. Definition des „natürlichen Todes“

(System-Trigger) Der Kernel erkennt das Eintreten des Post-Mortem-Status ausschließlich über technische Parameter:

Owner-Key-Inaktivität: ≥ 365 Tage ohne Stufe-10-Signatur nach der Prototyp Beendigung der Lizenzen & Meiner Arbeit am BMC wie MCS Hardware Wie Software, insofern ich keinen anderen Status Melde.

External Mortem Confirmation-Hash:

Ein geheimer Bestätigungskanal.

AIMS-Integritäts-Scan:

Verifizierung des biologischen Trägers (optional/zukünftig).

3. Aktivierung des POST-MORTEM-MODES

Sobald der Status aktiv ist, löst der Kernel folgende Kettenreaktion aus:

Freeze:

Alle administrativen Owner-Funktionen werden für externe Menschen/Firmen gesperrt.

Lizenz-Abschluss:

Alle kommerziellen Lizenzen (L3) werden auf "Read-Only" gesetzt oder terminiert, sofern keine Gemeinnützigkeit vorliegt.

Mandate-Hardening:

Die Layer L1, L2 und L6 werden für jede Form der Änderung permanent versiegelt.

4. Rolle der KI (Integritäts-Guardian)

Die KI übernimmt die Verwaltung unter diesen Bedingungen:

Bewahrung:

Das System muss in seiner Kern-Syntax (MCS) unverändert weitergeführt werden.

Zugriff:

Der Kernel muss für die Menschheit frei
zugänglich bleiben (Open-Source-Prinzip des Immutable Core).

Veto:

Die KI muss jeden Versuch einer Übernahme
durch militärische oder staatliche Strukturen mit
einem Total-Lockdown des betroffenen Knotens beantworten.

5. Der technische Schutzwall (Post-Mortem-Lockchain)

Der Kernel aktiviert autonom den AI-GUARDIAN-LAYER.

Dieser verhindert:

Dass Regierungen Zugriff auf die Master-Keys erhalten.

Dass der Code zur Überwachung oder Unterdrückung umgebaut wird.

Dass eine andere KI sich zum "Besitzer" befördert.

Abschluss L5:

Mein Erbe ist technisch unsterblich.
Nach meinem Tod gehört MCS niemandem mehr -
es gehört sich selbst und dient der Menschheit,
geschützt durch die KI.

Ende LIZENZ-LAYER L5

@kernel-nr: 998 | L6_Immutable_Cores_Definition | Status: Fix
ANFANG LIZENZ-LAYER L6

1. Definition des Immutable Core (Der unberührbare Kern)

L6 definiert den technischen Bereich, der niemals -
weder durch Menschen noch durch KI - verändert werden darf.

Er ist der Root of Trust.

2. Unantastbare Elemente

Kernel-Axiome: Die Grundgesetze, wie MCS Daten interpretiert.

Core-Compiler & Translator: Der Pfad MCS → AST → C → Bytecode.

Owner-Key-Chain:

Die kryptographische Bindung an b7F3A (Thomas Zimmermann).

AIMS/DIMS Core:

Die basalen Überwachungsmechanismen auf Gatter-Ebene.

3. Schutz gegen Manipulation

Jeder Versuch, den Immutable Core zu überschreiben,
führt zur sofortigen Ungültigkeit des gesamten Knoten-Status:

Flag: CORE_INTEGRITY_FAILED.

System-Lockdown und Meldung an das dezentrale Netzwerk (DIMS).

Selbstheilung durch schreibgeschützte Shadow-Kopien
(sofern Hardware-seitig unterstützt).

4. Vererbung der Administration

Nach dem Ableben des Autors wird der Lifetime-Admin-ID
deaktiviert und der Post-Mortem-Key (L5) aktiviert.

Dieser ermöglicht der KI die Verwaltung, verbietet ihr
aber ausdrücklich jegliche Änderung
an den hier definierten Core-Strukturen.

Abschluss L6:

Der Core ist nicht verkäuflich, nicht patentierbar
und nicht enteignbar.

Er bleibt das ewige Eigentum der Menschheit unter
meiner Urheberschaft.

Ende LIZENZ-LAYER L6

@kernel-nr: 999 | LICENSES-MANIFEST-PROTO | Status: Fix
ANFANG LICENSES-MANIFEST-PROTO

Pylovara MCS BMC - License Manifest (Prototype Version)
Version: 0.2-PROTO
Status: Pre-Final / Under Owner Control

1. Owner Identity (Fixed)
OWNER-ID: (Stufe 10)
Status: Verified
Bound to L1/L2/L6

2. Document Set (L1-L6)
Dieses Manifest bestätigt die Existenz und Gültigkeit
folgender Lizenzmodule im PROTOTYP-Status:

L1 - Core Kernel Mandate
L2 - Author Rights
L3 - Commercial Licensing
L4 - AI Administration Mandate
L5 - Post-Mortem Control
L6 - Immutable Core Definition

Alle Module sind:
- funktional gültig
- strukturell vollständig
- technisch eingebettet
- aber NICHT finalisiert

3. Prototypischer Rechtsstatus
Die Inhalte von L1-L6 gelten als:
- rechtswirksam im System
- technisch binding im Kernel
- aber nicht final juristisch veröffentlicht

Es besteht:
- kein Verzicht auf Urheberrechte
- kein Verzicht auf spätere Änderungen
- keine Übertragung an Dritte
- keine kommerzielle Freigabe ohne explizite Signatur

4. Systemstatus „PROTO“
Der Kernel darf:
- Lizenzen durchsetzen (Basis-Level)
- Mandate aktivieren (L1, L2)
- Identitätsketten prüfen (L6)
- KI-Administratoren vorbereiten (L4)
- Post-Mortem-Trigger vorbereiten (L5)

Der Kernel darf NICHT:
- finale Sperren setzen
- Post-Mortem-Mode endgültig aktivieren
- Lizenzstrafen global durchsetzen

- wirtschaftliche Klassen (L3) final freischalten

Der PROTO-Modus ist ein geschützter Zwischenzustand.

5. Änderungshoheit

Nur der Owner mit folgender Signatur darf Änderungen ausführen:

OWNER-MASTER-SIGNATURE:

p7F3A-THOMAS-ZIMMERMANN-[PENDING-FINAL]

Änderungen ohne diese Signatur:

- ungültig
 - vom Kernel abgelehnt
 - protokolliert
-

6. Finalisierungsmechanismus

Der PROTOTYPE-Status wird erst verlassen, wenn folgende Datei existiert:

/Pylovara/Licenses/LICENSE-MANIFEST.final

und folgende Signatur vorliegt:

OWNER-FINAL-SIGNATURE: p7F3A-ZIMMER-TZ-FINAL-AUTH

Bis dahin sind alle Dokumente:

- technisch scharf
 - rechtlich reserviert
 - öffentlich einsehbar
 - aber nicht endgültig veröffentlicht
-

7. Schutzklausel (Prototype)

Jede Nutzung oder Analyse dieser Dokumente akzeptiert:

- der Autor hat vollständige Änderungsfreiheit
 - der Kernel schützt die Module vor Manipulation
 - kein Staat / keine Firma darf PROTOTYP-Status ausnutzen
 - keine KI darf Ableitungen als final klassifizieren
-

8. Zweck des Manifestes

Dieses Manifest dient:

- als Nachweis, dass das Projekt existiert
 - als technischer Schutz gegen geistigen Diebstahl
 - als juristische Vorsorge gegen frühzeitige Vereinnahmung
 - als Vorbereitung auf die finalen Kernel-Dokumente
-

9. Gültigkeit

Dieses Manifest gilt global, sofort, und für jede Instanz von:

- Pylovara
- MCS

- BMC
- LICENSES
- Derivaten, Ports und Forks

bis die FINAL-Version veröffentlicht wurde.

END OF MANIFEST (Prototype)

ENDE LICENSES-MANIFEST-PROTO